浙江省市县联赛分组、抽签、选址与赛制优化模型

摘要

浙江省市县联赛拟由 11 个设区市、20 个县级市、33 个县共 64 个单位参赛，分为 16 个小组进行单循环比赛后决出 32 强淘汰晋级。赛事组织面临分组公平性、抽签随机性、选址合理性和赛制竞技保真度四类核心问题。本文围绕上述问题建立了从分组、抽签、选址到赛制评价的完整数学模型体系，主要工作如下。

分组方案：将分组建模为约束满足问题，定义 C3 冲突对数 F_1、实力标准差 F_2 和实力极差 F_2' 三项指标，提出贪心优先填充、蛇形轮转、ILP-C3 最小化和 ILP 均衡优化四种策略。利用 Hall 婚配定理严格证明满足 C1、C2 硬约束的分组方案恒存在，并由 ILP 均衡方案实现 $F_1 = 0$、$F_2 = 0.484$、$F_2' = 1$ 的最优均衡，各组实力极差仅 1 个实力分。

抽签机制：提出前瞻最小费用流辅助抽签法，核心思想是在每步抽签前通过求解剩余最小费用流，仅保留能延续全局最优分配的小组进入抽签盆，从源头上杜绝死锁。分析了传统贪心抽签的死锁机制与触发条件（1 000 种子实验中死锁率 9.9%），前瞻流方案则实现零失败和 $F_1 = 0$。在 100 个随机行政拓扑的广义检验中，前瞻网络流在全部 300 次抽签中失败率为零（贪心死锁率 13%），当 C3 零冲突在数学上不可实现时自动转为最小冲突分配。

比赛地点：将选址建模为带主场禁止约束的设施选址与容量指派 ILP 模型。基准方案（haversine 距离）得到 78 01 km 总旅行距离的最优选址；进一步构造公路时间、铁路时间、综合时间、人口加权和影响力 Top20 候选集五种成本模型检验选址稳健性，推荐以综合时间模型为主要选址依据。

赛制优化：基于 Bradley-Terry 模型和层内正态随机实力赋值进行 5 000 次赛事仿真。结果表明，将单循环改为双循环可使 Spearman 相关系数由 0.471 提升至 0.564，Top-32 晋级准确率由 69.2%提升至 74.6%。结合仿真诊断，提出种子分档、同组回避交叉对阵、小组第三附加赛和双败淘汰试点等增强方案及分阶段实施路径。

关键词：约束满足问题；前瞻最小费用流；Hall 婚配定理；设施选址；Bradley-Terry 模型；赛制仿真

1 问题重述

1.1 问题背景

2025 年江苏城市足球联赛（“苏超”）引发了全社会的广泛关注。拟议中的浙江省市县联赛有浙江省 11 个副省级城市和地级市（合称市级），20 个县级市和 33 个县（合称县级），共 64 个单位参加。比赛分为两个阶段：第一阶段为小组赛，参赛队分为 16 个小组，每个小组 4 支队伍，进行单循环比赛，小组前两名进入下一阶段；第二阶段为淘汰赛，32 支队伍经过 5 轮比赛，决出冠军。

1.2 分组约束

体育赛事分组应遵循公平、透明的原则，本次赛事分组还需遵循以下原则：

(**C1**) 各市级队应分配在不同小组；

(**C2**) 市级队不与该市代管县的县级队分配在同一小组；

(**C3**) 同一市代管县的县级队应尽量不分配在同一小组。

1.3 行政区划数据

浙江省 11 个设区市及其代管县市的数量分布如表 1 所示。

表 1: 浙江省各市球队分布

设区市	市级队	代管县级市	代管县	县级队合计
杭州	杭州市	建德市	桐庐县、淳安县	3
宁波	宁波市	余姚市、慈溪市	象山县、宁海县	4
温州	温州市	瑞安市、乐清市、龙港市	永嘉县、平阳县等 5 县	8
嘉兴	嘉兴市	海宁市、平湖市、桐乡市	嘉善县、海盐县	5
湖州	湖州市	—	德清县、长兴县、安吉县	3
绍兴	绍兴市	诸暨市、嵊州市	新昌县	3
金华	金华市	兰溪市、义乌市、东阳市、永康市	武义县、浦江县、磐安县	7
衢州	衢州市	江山市	常山县、开化县、龙游县	4
舟山	舟山市	—	岱山县、嵊泗县	2
台州	台州市	温岭市、临海市、玉环市	三门县、天台县、仙居县	6
丽水	丽水市	龙泉市	青田县、缙云县等 7 县	8
合计	11	20	33	53

1.4 待解决的问题

问题 1 给出若干符合要求的分组方案，并从不同维度比较它们的优劣。

问题 2 设计一种可行的抽签方案，使抽签得到的分组方案满足相应要求，并具有较好的性质。

问题 3 选择八个市或县作为小组赛的比赛地点（每个地点进行两个小组的比赛），考虑公平性、赛事影响和普及面、交通便利性。

问题 4 基于数学模型和定量分析，给出对赛制的合理化建议。

2 基本假设与符号说明

2.1 基本假设

(1) 题目未给出各队真实竞技水平，采用层级赋权法近似实力：市级队 $s = 3$，县级市队 $s = 2$，县队 $s = 1$。该赋值反映行政层级与资源禀赋的一般差异，作为全文公平性和均衡分析的基准。后文分别通过问题 2 的随机拓扑实验和问题 4 的层内正态随机实力模型（式 (27)）检验核心结论的稳健性。

(2) 基准选址模型采用球面坐标下的大圆距离（haversine 公式）计算旅行距离；扩展模型进一步引入公路、铁路旅行时间矩阵。

(3) 城市影响力和赛事普及面用常住人口、交通可达性等公开指标近似刻画。

(4) 单场胜负结果服从 Bradley-Terry 模型，强制分出胜负（无平局）。真实足球比赛存在平局和积分 3/1/0 制，本模型将其简化为必分胜负的 BT 概率，会高估单场比赛的信息量，因此本文只比较同一简化模型下不同赛制的相对差异。小组同分排序中的进球数用 Poisson 随机量近似。

2.2 符号说明

表 2: 主要符号说明

符号	含义
x_{ij}	球队 i 是否分入小组 j 的 0-1 变量
n_{cj}	城市 c 在小组 j 中的县级队数量
s_i	球队 i 的实力分值（市=3，县级市=2，县=1）
S_j	小组 j 的实力总和
F_1	C3 冲突对数（同市县级队同组的无序对数）
F_2	16 组实力标准差
F_2'	16 组实力极差（最大减最小）
$\mathcal{G}_j$	第 j 个小组的球队集合
y_k	地点 k 是否被选为比赛场地的 0-1 变量
z_{gk}	小组 g 是否安排到场地 k 的 0-1 变量
d_{ik}	球队 i 所在地到场地 k 的距离
c_{ik}	球队 i 到场地 k 的等效成本
τ	约束紧度 $= \max(n_i)/15$

3 问题 1：分组方案设计与比较

3.1 数学模型

3.1.1 决策变量与基本约束

定义 0-1 决策变量 $x_{ij} \in \{0,1\}$，表示球队 i 是否分入小组 j（$i = 1,2,\ldots,64$，$j = 1,2,\ldots,16$）。

每支球队恰好进入一个小组，每个小组恰好容纳 4 支球队：

$$\sum_{j=1}^{16} x_{ij} = 1, \quad \forall i; \qquad \sum_{i=1}^{64} x_{ij} = 4, \quad \forall j \tag{1}$$

C1（市级队不同组）：令 M 为 11 支市级队的集合，则

$$\sum_{i \in M} x_{ij} \le 1, \quad \forall j \tag{2}$$

C2（市级队不与本市县级队同组）：令 $m(c)$ 为城市 c 的市级队，C_c 为城市 c 下辖县级队集合。则

$$x_{m(c),j} + x_{i,j} \leq 1, \quad \forall i \in C_c,\ \forall c \in \{1, \ldots, 11\},\ \forall j \tag{3}$$

C3（同市县级队尽量不同组）：C3 是软约束。若将其严格化，则

$$\sum_{i \in C_c} x_{ij} \leq 1, \quad \forall c,\ \forall j \tag{4}$$

3.1.2 容量与可行域分析

16 组总容量为 $16 \times 4 = 64$，恰好容纳全部球队。其中 11 组含有市级队（各剩 3 个县级名额），5 组无市级队（各 4 个县级名额），县级名额总计 $11 \times 3 + 5 \times 4 = 53$，恰好等于县级队总数。

C3 可行性：各市县级队需尽量分散到不同组。关键约束来自县级队最多的两个城市——温州（8 支）和丽水（8 支）。每市仅需避开本市市级队所在的 1 个组，可用组数为 15。由于 $8 \leq 15$，单个城市层面不存在组数瓶颈。下文先用 Hall 婚配定理证明 C1+C2 硬约束恒可行，再用 ILP 求解结果给出本题 C3 零冲突的构造性证据。

3.1.3 基于 Hall 婚配定理的严格证明

上述直观论证可以由 Hall 婚配定理给出严格的形式化证明。

定义 3.1 (约束二部图). *构造二部图 $G = (U \cup V, E)$，其中：*

- *U：53 支县级队的集合；*
- *V：53 个县级队槽位（有市级队的组各 3 个，无市级队的组各 4 个）；*
- *$(u, v) \in E$ 当且仅当：县级队 u 所属市的市级队不在槽位 v 所在的组中（即不违反 C2 约束）。*

在该二部图中，完美匹配对应一个满足 C1 和 C2 的完整分组方案（注意：该二部图仅编码 C2 禁入关系，不含 C3 同市互斥约束；C3 的可满足性由下文推论给出论证）。

命题 3.2 (C1+C2 可行性). *固定任意一种满足 C1 的市级队落位。对任意县级队子集 $S \subseteq U$，记 $B(S)$ 为 S 中球队所属城市的集合。若 $|B(S)| = 1$，则 S 中所有队伍只共同禁入 1 个含市级队的小组，邻域至少包含其余 15 个小组的 50 个县级槽位，因此 $|N(S)| = 50 \geq |S|$。若 $|B(S)| \geq 2$，则任意一个县级槽位至多只会被其所属城市的县级队禁入，但一定可由 S 中其他城市的县级队使用，所以 $N(S) = V$，有 $|N(S)| = 53 \geq |S|$。于是 Hall 条件对所有 $S \subseteq U$ 成立，二部图 G 存在完美匹配，即满足 C1 和 C2 的完整分组方案一定存在。*

推论 3.3 (本题 C3 零冲突的构造性证据). *C3 是软约束，Hall 证明本身不包含“同市县级队互斥”条件。本文将 C3 冲突对数 F_1 纳入 ILP 目标，并用 CBC 求解得到 $F_1 = 0$ 的方案 D; 附录 A 列出完整分组。该结果构成了本题数据下 C3 可完全满足的构造性证据。若换成其他行政拓扑，C3 零冲突未必可行，因此问题 2 中将 C3 作为最小费用目标而不是硬约束处理。*

Hall 定理保证了全局解的存在性，但不保证任意贪心决策路径都能通向可行解。问题 2 中将详细分析贪心抽签算法在何种条件下会陷入死锁，并在此基础上设计一种“事前筛选”的前瞻抽签机制。

3.2 评价指标体系

指标 1：C3 冲突对数 F_1

$$F_1 = \sum_{c=1}^{11} \sum_{j=1}^{16} \binom{n_{cj}}{2} \tag{5}$$

其中 $n_{cj} = \sum_{i \in C_c} x_{ij}$ 为城市 c 分入小组 j 的县级队数量。$F_1 = 0$ 表示 C3 完全满足，越小越好。

指标 2：实力标准差 F_2 与极差 F_2'

各组实力总和 $S_j = \sum_i s_i x_{ij}$（市级 $s = 3$，县级市 $s = 2$，县 $s = 1$），总实力 $\sum s_i = 106$，组均值 $\bar{S} = 6.625$。

$$F_2 = \sqrt{\frac{1}{16} \sum_{j=1}^{16} (S_j - \bar{S})^2}, \qquad F_2' = \max_j S_j - \min_j S_j \tag{6}$$

两者均越小越好，反映各组实力是否均衡。

3.3 分组方案生成算法

3.3.1 方案 A：贪心优先填充

思想：按县级队数量从多到少依次处理各市，每支县级队贪心选择 C3 冲突最小且人数最少的组。

算法 1 方案 A：贪心优先填充

输入：11 个城市的市级队和县级队，16 个小组，组容量 4
输出：分组方案 $\{\mathcal{G}_1, \ldots, \mathcal{G}_{16}\}$
初始化 $\mathcal{G}_1, \ldots, \mathcal{G}_{16} \leftarrow \emptyset$
按县级队数降序排列城市，得到序列 C
for $r = 1$ **to** 11 **do**

```
    将城市 C_r 的市级队放入 G_r，记录禁入组 m(C_r) = r
end for
for 城市 c ∈ C do
    随机打乱城市 c 的县级队序列 T_c
    for 县级队 t ∈ T_c do
        A ← {j : |G_j| < 4, j ≠ m(c)}
        对每个 j ∈ A 计算 same_cj：组 j 中是否已有城市 c 的县级队
        选择 j* = arg min_{j∈A}(same_cj, |G_j|)；若并列则随机选取
        将 t 放入 G_{j*}
    end for
end for
return {G_j}_{j=1}^{16}
```

3.3.2 方案 B：蛇形轮转分配

思想：借鉴体育赛事中的“蛇形排列”，将各市县级队在小组间按轮转方式分散放置。

算法 2 方案 B：蛇形轮转分配

```
1: 输入：城市序列 C，16 个小组，组容量 4
2: 输出：分组方案 {G_1, ..., G_16}
3: 初始化小组并按算法 1 第 3–6 行放置市级队
4: 构造蛇形序列 P = (1, 2, ..., 16, 16, 15, ..., 1, ...)，令指针 p ← 1
5: for 城市 c ∈ C do
6:     随机打乱城市 c 的县级队序列 T_c，并置 U_c ← ∅
7:     for 县级队 t ∈ T_c do
8:         placed ← false
9:         for ℓ = 1 to 16 do
10:            j ← P_p，p ← p + 1
11:            if j ≠ m(c) and |G_j| < 4 and j ∉ U_c then
12:                将 t 放入 G_j，U_c ← U_c ∪ {j}，placed ← true
13:                break
14:            end if
15:        end for
16:        if not placed then
17:            在满足 C2 且未满的组中顺延放置 t；优先选择 j ∉ U_c 的组
18:        end if
19:    end for
20: end for
```

21: **return** $\{\mathcal{G}_j\}_{j=1}^{16}$

3.3.3 方案 C：ILP-C3 最小化

以 F_1 最小化为目标，C1 和 C2 为硬约束，建立整数线性规划模型（ILP）：

$$\begin{aligned} \min \quad & \sum_{c=1}^{11}\sum_{j=1}^{16}\binom{n_{cj}}{2} \\ \text{s.t.} \quad & \text{式}(1),(2),(3) \end{aligned} \tag{7}$$

代码实现中使用 PuLP 调用 COIN-OR CBC 求解器求解。

3.3.4 方案 D：ILP 均衡优化

实际求解时先将 C3 冲突对数作为高权重目标，再最小化组间实力极差。令同市县级队配对变量 u_{pj} 表示配对 p 是否同落入组 j，并引入 $S_{\max}$、$S_{\min}$ 约束各组实力和，则目标为

$$\begin{aligned} \min \quad & w_1\sum_{p,j}u_{pj} + w_2(S_{\max}-S_{\min}) \\ \text{s.t.} \quad & \text{式}(1),(2),(3) \\ & u_{pj} \geq x_{a_p,j} + x_{b_p,j} - 1, \quad \forall p=(a_p,b_p), j \\ & S_{\max} \geq S_j, \quad S_{\min} \leq S_j, \quad \forall j. \end{aligned} \tag{8}$$

这里我们取 $w_1 = 10, w_2 = 1$，求解结果自然达到 $F_1 = 0$，同时将实力极差压缩到 1。

3.4 实验结果与方案对比

四种方案的评价指标对比如表 3 所示。附录 A 列出方案 A–D 的完整分组，便于直接核查各方案的约束满足情况和实力分布。

表 3: 四种分组方案的评价指标对比

方案	F_1（C3 冲突对数）	F_2（实力 σ）	F_2'（极差）
方案 A：贪心优先填充	0	0.927	3
方案 B：蛇形轮转分配	0	1.111	4
方案 C：ILP-C3 最小化	0	0.992	3
方案 D：ILP 均衡优化	0	**0.484**	**1**

分析：

(1) **C3 满足度**：四种方案均实现了 $F_1 = 0$，验证了“C3 在本题数据下可完全满足”的预判。

(2) **实力均衡度**：ILP 均衡方案（方案 D）表现最优，$F_2 = 0.484$ 仅为贪心方案（0.927）的 52%，$F_2' = 1$ 意味着各组实力极差仅为 1 个实力分，实现了高度的竞技公平。

(3) **方案推荐**：若优先追求公平性，应选方案 D（ILP 均衡）；若兼顾可解释性和计算效率，方案 A（贪心）是合理的次优选择。

4 问题 2：抽签方案设计

4.1 设计目标

抽签方案需满足以下性质：

(1) **可行性**：抽签结果必须满足 C1、C2 硬约束，并尽量满足 C3。

(2) **随机性**：在约束允许范围内保留尽可能多的随机选择；前瞻流方案先随机抽队，再从安全小组集合中随机抽取小组。

(3) **透明性**：抽签流程公开，每一步可审计。

(4) **鲁棒性**：不会因随机因素陷入死锁（无解）。

4.2 贪心死锁与前瞻抽签思想

4.2.1 朴素贪心抽签

在介绍完整抽签方案之前，先分析一种朴素贪心策略的死锁问题。该策略与问题 1 方案 A 类似，具体流程如算法 3 所示。

算法 3 朴素贪心抽签

输入：球队数据，随机种子
输出：分组方案或失败标记 $\perp$
随机将 11 支市级队分配到 16 组中的 11 个不同组，并记录各城市禁入组 $m(c)$
按县级队数量从多到少排列城市，得到序列 C
for 城市 $c \in C$ **do**
    随机打乱城市 c 的县级队序列 T_c
    for 县级队 $t \in T_c$ **do**
        $A \leftarrow \{j : |\mathcal{G}_j| < 4,\ j \neq m(c)\}$
        if $A = \emptyset$ **then**
            return $\perp$

```
        end if
        对每个 j ∈ A 计算 C3 冲突标记 same_cj
        选择 j* = arg min_{j∈A}(same_cj, |G_j|)；若并列则随机选取
        将 t 放入 G_{j*}
    end for
end for
return {G_j}_{j=1}^{16}
```

该策略不包含任何回溯机制，一旦某支队伍无合法候选组，算法即宣告失败。

4.2.2 全局可行性与贪心死锁的区别

问题 1 中已由 Hall 婚配定理证明：满足 C1 和 C2 的分组方案恒存在。然而，贪心策略每一步只看当前的局部最优，一旦放入便不再调整。这种不可逆的局部决策可能导致后续队伍的合法位置被提前占满，使算法陷入”死锁”。

4.2.3 死锁反例

以 seed = 0 为例，可以稳定复现一个死锁场景。市级队随机分配结果如表 4 所示。

表 4: 死锁反例中的市级队分配（seed=0）

组号	市级队	组号	市级队
G1	金华市	G7	绍兴市
G2	丽水市	G8	温州市
G3	杭州市	G9	宁波市
G4	湖州市	G10	台州市
G5	衢州市	G11	**舟山市**
G6	嘉兴市	G12–G16	无市级队

算法按县级队数量降序依次处理温州 (8)→ 丽水 (8)→ 金华 (7)→ 台州 (6)→ 嘉兴 (5)→ 衢州 (4)→ 宁波 (4)→ 湖州 (3)→ 绍兴 (3)→ 杭州 (3)，共放置 52 支县级队。舟山市仅 2 支县级队，排在最后。处理过程中，舟山的岱山县已被分配到 G7，而 G11（舟山市所在组）仅放入 3 队（舟山市+其他市的县级队），恰好留有 1 个空位。

当轮到舟山的最后 1 支队伍嵊泗县时：

- G11（舟山市所在组）：C2 禁入，不可放入；
- G1–G10、G12–G16：全部恰好满员（4 队）。

唯一有空位的组是禁入组，全部合法位置均被占用——算法陷入不可恢复的死锁。这是因为贪心策略在处理前 10 个市的 52 支县级队时均匀填充了所有组，不预判后续城市的需求，未为最后处理的舟山市保留必要的灵活性。

4.2.4 死锁的触发条件与频率

1 000 个固定随机种子实验中，朴素贪心策略死锁率为 9.90%。死锁更易在以下条件下触发：

(1) 县级队数量少的城市被排在最后处理；

(2) 前续城市的贪心放置恰好填满了所有非禁入组；

(3) 最后待分配队伍的 C2 禁入组是唯一有空位的组。

广义蒙特卡洛实验表明，死锁并非由约束系统本身无解导致，而是由抽签路径的局部不可逆性导致：在 100 个随机拓扑、每个拓扑 3 个配对种子的实验中，朴素贪心总死锁率为 13.0%，而前瞻网络流失败率为 0（见表 6）。

4.3 前瞻最小费用流辅助抽签法

朴素贪心只依据当前局部信息决策，遇到死锁后只能整体作废重抽。若用于真实体育抽签仪式，已经公布的结果不宜被撤回，也不宜在事后调整已公布分组。因此采用事前筛选思想：每抽出一支县级队后，计算机先判断哪些小组能够通向全局最优的剩余分配，再把这些安全小组编号放入现场小组盆中进行物理抽取。

4.3.1 状态变量

设当前抽签状态下：

- r_j：小组 j 的剩余名额；
- $m(c)$：城市 c 的市级队所在小组，即城市 c 县级队的 C2 禁入组；
- a_{cj}：城市 c 已有多少支县级队在小组 j 中；
- b_c：城市 c 尚未抽出的县级队数量。

当前 C3 冲突对数为

$$F_1^{\text{cur}} = \sum_c \sum_{j=1}^{16} \binom{a_{cj}}{2}. \tag{9}$$

若未来还要把 y_{cj} 支城市 c 的县级队放入小组 j，则新增 C3 冲突为

$$\Delta_{cj}(y_{cj}) = \binom{a_{cj} + y_{cj}}{2} - \binom{a_{cj}}{2}. \tag{10}$$

因此，C3 不是普通的边可行性约束，而是城市 – 小组层面的凸费用。

4.3.2 剩余分配的最小费用流模型

对当前抽出的县级队 t（所属城市为 c_t），假设尝试将其放入小组 g。若 $g = m(c_t)$ 或 $r_g = 0$，则该组直接排除；否则临时放入 t，更新 $r_g, a_{c_t g}, b_{c_t}$，并对剩余队伍求解如下最小费用流：

$$
\begin{aligned}
\min \quad & \sum_{c} \sum_{j=1}^{16} \left[\binom{a_{cj} + y_{cj}}{2} - \binom{a_{cj}}{2} \right] && (11) \\
\text{s.t.} \quad & \sum_{j=1}^{16} y_{cj} = b_c, \quad \forall c, && (12) \\
& \sum_{c} y_{cj} = r_j, \quad \forall j, && (13) \\
& y_{cj} = 0, \quad j = m(c), && (14) \\
& y_{cj} \in \mathbb{Z}_{\geq 0}.
\end{aligned}
$$

网络构造如下。源点连向每个城市节点 c，容量 b_c、费用 0；城市 c 连向小组满足 $j \neq m(c)$ 的 j 的边用单位容量平行边线性化式 (10)，第 ℓ 条边的费用为 $a_{cj} + \ell - 1$；小组 j 连向汇点，容量 r_j、费用 0。若最大流量等于 $\sum_c b_c$，则剩余抽签可完成；其最小费用即为未来最少还需增加的 C3 冲突数。

为提高实现效率，程序先用最大流检查 $F_1 = 0$ 是否仍可行：只保留 $a_{cj} = 0$ 的城市 – 小组边，容量为 1；若该最大流满流，则该候选组的最终 F_1 可保持为 0，无需调用完整最小费用流。只有零冲突不可延续时，才退回到完整的最小费用流模型。

4.3.3 现场抽签流程

算法 4 前瞻最小费用流辅助抽签法

输入： 11 支市级队、53 支县级队、16 个小组、组容量 4
输出： 完整分组方案
阶段 1：市级队定位
从市级队盆中随机抽取 11 支市级队，并随机抽取 11 个不同小组放入，记录 $m(c)$
阶段 2：县级队前瞻抽签
将 53 支县级队混合放入县级队盆，随机抽出一支队 t，记其城市为 c_t
for 每个小组 $g = 1, \ldots, 16$ **do**
    if $g = m(c_t)$ **or** $r_g = 0$ **then**
        标记 g 为不可用
    else
        临时将 t 放入 g，求解剩余队伍的最大流/最小费用流
        if 剩余最大流不能满流 **then**

```
            标记 g 为不可完成
        else
            记录该选择对应的最小最终 C3 冲突数 Φ(g)
        end if
        撤销临时放置
    end if
end for
令 Φ* = min_g Φ(g)，安全小组集 A(t) = {g : Φ(g) = Φ*}
将 A(t) 中的小组编号放入现场小组盆，随机抽出正式小组
重复阶段 2，直到 53 支县级队全部抽取完毕
return 16 个小组
```

这里的最小费用最大流在我们的代码中使用 Bellman-Ford 和 Dinic 算法实现。

4.3.4 可行性保证

命题 4.1 (前瞻抽签不死锁). *若当前部分抽签状态存在至少一个满足 C1、C2 的完整延拓，则算法 4 在当前抽出的球队 t 上得到的安全小组集 $A(t)$ 非空。若从 $A(t)$ 中任取一个小组正式放置 t，则新状态仍存在满足 C1、C2 的完整延拓；并且该选择在所有可完成选择中保持最终 F_1 的最小可实现值。*

证明。设当前状态存在一个完整延拓。由于 t 是尚未放置的球队之一，在该延拓中 t 必被放入某个小组 $g^\star$。该小组必然满足容量约束且不等于 t 所属城市的 C2 禁入组；固定选择 $t \to g^\star$ 后，其他未放置球队仍可按照该延拓完成分配。因此，算法枚举小组时，$g^\star$ 对应的剩余流模型能够满流，$\Phi(g^\star)$ 有定义，故至少存在一个可完成候选组，$A(t)$ 非空。

对任意 $g \in A(t)$，其定义已经要求固定 $t \to g$ 后的剩余流模型可满流，因此正式选择该组后仍存在完整延拓，不会进入死锁状态。同时，$A(t)$ 只保留使 $\Phi(g)$ 达到最小值 $\Phi^* = \min_g \Phi(g)$ 的候选组，所以该选择在所有可完成选择中保持最终 C3 冲突数最小。由归纳法，从初始存在完整延拓的状态出发，每一步之后仍存在完整延拓，直到 53 支县级队全部放置完成，算法均不会出现无可用小组。 □

该命题说明，前瞻流不是事后调整或整体重抽，而是在每一步把会导致未来死锁或劣化 C3 目标的危险小组提前从现场小组盆中剔除。由于 C3 以费用而非硬禁令进入模型，当某些泛化数据无法实现 $F_1 = 0$ 时，该方法仍会自动选择 C3 冲突数最小的可完成路径。

4.4 策略对比实验

为验证新抽签法的效果，本文在同一批 1 000 个随机种子上比较朴素贪心与前瞻最小费用流。朴素贪心代表“不预判未来”的传统现场抽签，前瞻流代表本文推荐的事前

筛选机制。由于前瞻流每步需要求解多个流模型，计算成本高于贪心策略，但本题规模较小，仍能满足现场实时筛选需求。

表 5: 朴素贪心与前瞻流的 1 000 个配对种子实验对比

指标	朴素贪心	前瞻最小费用流
成功率	90.1%	**100%**
死锁/失败率	9.9%	**0%**
$P(F_1 = 0)$	100%*	**100%**
$\mathbb{E}[F_1]$	0*	**0**
$\mathbb{E}[F_2]$	1.139	**1.129**
$\mathbb{E}[F_2']$	4.01	**3.98**
平均前瞻检查次数	–	582.7
平均计算时间/次	0.0011s	0.2153s

*朴素贪心的 F_1, F_2, F_2' 仅统计成功（未死锁）的样本。

实验结果表明：前瞻最小费用流在样本中实现了零失败和 $F_1 = 0$，同时不需要撤回已经公布的抽签结果。其代价是计算量较高：1 000 次实验中平均每次抽签进行了 582.7 次前瞻可行性检查，其中 58.6 次需要调用完整最小费用流模型；单次抽签平均耗时约 0.22 秒，仍可满足现场“抽队伍 – 显示可用小组 – 抽小组”的实时需求。

4.5 边际均匀性检验

前瞻流会根据剩余可完成性动态删去部分候选组，因此需要检查该筛选机制是否明显破坏抽签的随机性。本文采用边际均匀性检验：对每支球队 t，统计其在 1 000 次前瞻流抽签中落入 16 个小组的频数 $O_{t1}, \ldots, O_{t,16}$。在零假设 H_0 下，球队 t 落入各小组的边际概率相等，即期望频数为 $E_{tj} = 1000/16$。检验统计量为

$$\chi_t^2 = \sum_{j=1}^{16} \frac{(O_{tj} - E_{tj})^2}{E_{tj}}, \quad df = 15. \tag{15}$$

逐队卡方检验中，前瞻流有 59/64 支球队通过 $\alpha = 0.05$ 显著性水平；考虑到同时进行了 64 次检验，采用 Bonferroni 校正后显著性阈值为 $0.05/64 \approx 0.0008$，此时 64 支球队全部通过。进一步将 64 个 p 值与 $U(0,1)$ 分布做 Kolmogorov–Smirnov 检验，得到 $D = 0.0947, p = 0.5816$，未发现 p 值整体偏离均匀分布的证据。该结果说明，前瞻筛选虽然会剔除危险小组，但在本题样本规模下未表现出系统性的边际分组偏差。

4.6 广义蒙特卡洛鲁棒性检验

前述实验固定于浙江省实际行政区划。为检验抽签死锁机制是否依赖特定数据，本文构造 100 个随机行政拓扑，并在每个拓扑上运行 3 个配对随机种子。实验仅比较两类

模型：基础贪心抽签与前瞻网络流分配模型。为衡量问题难度，引入约束紧度（tightness）τ：

$$\tau = \frac{\max_i n_i}{15} \tag{16}$$

其中 $\max_i n_i$ 为县级队最多的市的县级队数，15 为排除 C2 禁入组后的可用组数。τ 的物理意义是：最紧张的城市，其县级队需求占可用组数的比例。若 $\tau < 1$，C3 有望完全满足；若 $\tau = 1$，C3 恰好满足但零冗余；若 $\tau > 1$，C3 零冲突不可能完全满足。浙江省的 $\tau = 8/15 = 0.533$，属于中等偏紧水平。

随机拓扑生成时，先设定市级队数量 $k \in [5, 16]$，再将 $64 - k$ 支县级队按不同集中度分配给各市。县级队分布通过 Dirichlet 分布控制：均匀型采用确定性等分，中等变异采用对称 Dirichlet 参数 $\alpha = (2, \ldots, 2)$，集中型采用少数城市 $\alpha = 3$、多数城市 $\alpha = 0.5$ 的非均匀参数。每个拓扑上只运行基础贪心和前瞻网络流两个模型。前瞻网络流在 $\tau \leq 1$ 时检验能否维持 $F_1 = 0$，在 $\tau > 1$ 时则检验其是否能自动转为最小 C3 冲突分配。

表 6: 不同紧度区间下的基础贪心与前瞻网络流对比（100 个随机拓扑）

紧度区间	拓扑数	贪心死锁率	贪心 $P(F_1{=}0)$*	流失败率	流 $P(F_1{=}0)$	流 $\mathbb{E}[F_1]$
中等（$\tau \leq 0.5$）	41	22.0%	100.0%	0.0%	100.0%	0.00
偏紧（$0.5 < \tau \leq 0.7$）	17	3.9%	100.0%	0.0%	100.0%	0.00
紧张（$0.7 < \tau \leq 1.0$）	16	14.6%	97.6%	0.0%	100.0%	0.00
超紧（$\tau > 1.0$）	26	3.8%	0.0%	0.0%	0.0%	11.50

*贪心 $P(F_1{=}0)$ 仅在未死锁的成功样本上统计；当 $\tau > 1$ 时，C3 零冲突在数学上不可实现。

表 6 说明，基础贪心在各紧度区间均可能死锁，且其死锁率还受到市级队数量、县级队集中度和前序填充路径共同影响，因此 τ 不应被解释为单一的死锁概率模型。相比之下，前瞻网络流在全部 300 次抽签中失败率为 0；当 $\tau \leq 1$ 时均保持 $F_1 = 0$，当 $\tau > 1$ 导致 C3 零冲突不可实现时，算法仍能给出可完成的较小冲突方案，平均 $F_1 = 11.50$。

5 问题 3：比赛地点选择

5.1 数学模型：设施选址与带容量指派

题目要求从 64 个市县中选取 8 个作为小组赛承办地，每个场地承办 2 个小组。该问题包含两层决策：选哪些地点开放（选址），以及每个小组去哪个场地（指派）。由于场地集合不同时最优指派也不同，因此将其统一为一个 ILP 模型同时优化。此外，为避免球队在本地比赛获得主场优势，加入主场禁止约束。

5.1.1 成本矩阵

ILP 模型只要求给定球队 i 到候选场地 k 的成本矩阵 c_{ik}。在基准模型中，成本直接取 haversine 球面距离：

$$c_{ik} = d_{ik}^{\text{hav}}, \tag{17}$$

其中 d_{ik}^{hav} 为球队所在地到候选场地的球面距离。主场公平不放入成本函数，而作为硬约束处理：若小组中包含地点 k 的本地球队，则该小组不能安排到地点 k。后续敏感性分析保持同一 ILP 结构，仅替换 c_{ik} 为公路时间、铁路时间、综合时间或人口加权时间，并通过 Top20 候选集体现城市影响力约束。

5.1.2 ILP 模型

定义决策变量：

- $y_k \in \{0,1\}$：地点 k 是否被选为比赛场地（$k = 1, \ldots, 64$）；
- $z_{gk} \in \{0,1\}$：小组 g 是否安排到场地 k（$g = 1, \ldots, 16$）。

小组 g 若安排到场地 k，其总成本为 $C_{gk} = \sum_{i \in G_g} c_{ik}$。

$$\min \quad \sum_{g=1}^{16} \sum_{k=1}^{64} z_{gk} C_{gk} \tag{18}$$

$$\text{s.t.} \quad \sum_{k=1}^{64} y_k = 8 \tag{19}$$

$$\sum_{k=1}^{64} z_{gk} = 1, \quad \forall g \tag{20}$$

$$z_{gk} \le y_k, \quad \forall g, k \tag{21}$$

$$\sum_{g=1}^{16} z_{gk} = 2y_k, \quad \forall k \tag{22}$$

$$z_{gk} = 0, \quad \text{若地点 } k \in \mathcal{G}_g \tag{23}$$

约束 (19) 确保恰好选择 8 个场地；(20) 确保每个小组分配到恰好一个场地；(21) 确保只能分配到已选场地；(22) 确保每个选定场地恰好承办 2 个小组；(23) 为主场禁止约束，避免任何球队在本地比赛。

5.2 实验结果

由于问题 1 推荐方案 D（ILP 均衡优化）为最优分组，选址以方案 D 的分组为基础进行求解。这里采用 PuLP 库调用 CBC 求解器求解上述 ILP 模型。求解结果如表 7 所示。

表 7: ILP 最优选址结果（基于方案 D 分组）

场地	经度	纬度	承办小组	平均距离/队
余姚市	121.15°E	30.04°N	组 2、组 3	158 km
诸暨市	120.24°E	29.71°N	组 5、组 6	113 km
嵊州市	120.83°E	29.56°N	组 12、组 16	145 km
东阳市	120.24°E	29.29°N	组 4、组 11	101 km
武义县	119.81°E	28.90°N	组 8、组 10	114 km
磐安县	120.45°E	29.06°N	组 1、组 15	128 km
仙居县	120.73°E	28.85°N	组 13、组 14	112 km
缙云县	120.09°E	28.66°N	组 7、组 9	105 km

选址结果分布在浙中偏南区域（纬 28.66°–30.04°N，经 119.81°–121.15°E），兼顾了浙北（余姚）、浙中（诸暨、嵊州、东阳、武义、磐安、缙云）和浙南（仙居）。主场禁止约束生效后，最近单队距离由 0 提升到 10 km，避免了本地球队直接获得主场优势。需指出的是，纯距离模型仍偏向省域几何中心，未必自动选择杭州、宁波、温州等核心城市；因此下文进一步引入交通时间、人口加权和影响力候选集。

5.3 多维成本模型对比

仅用直线距离会低估海岛、山区和跨水域交通成本，因此进一步构造六类选址实验：haversine 距离、公路旅行时间、铁路旅行时间、公铁平均综合时间、人口加权公路时间，以及影响力 Top20 候选集下的综合时间模型。表 8 给出同一 ILP 模型在不同成本定义和候选集策略下的求解结果。

表 8: 不同成本模型下的 ILP 选址结果对比

成本模型	总成本	单位	平均/队	对应 haversine 距离
haversine	7 801	km	121.9	7 801 km
公路时间	8 372	min	130.8	8 220 km
铁路时间	6 524	min	101.9	8 531 km
综合时间	7 731	min	120.8	8 484 km
人口加权	662 682	万人 ·min	10 354.4	9 119 km
综合时间（影响力 Top20）	8 130	min	127.0	8 990 km

多维模型显示，不同成本定义会明显改变场地集合：haversine 与综合时间模型的场地交集为 2/8，而综合时间与影响力 Top20 综合时间模型的交集为 5/8。诸暨市、杭州市、义乌市、绍兴市等地点在多个模型中反复出现，说明这些地点在交通可达性和赛事

影响力之间具有较强稳健性。若赛事组织者只追求总旅行距离，可采用 haversine 或公路时间模型；若更重视真实交通体验，应优先采用综合时间模型；若更重视观众覆盖和传播效果，则可采用人口加权模型或影响力 Top20 候选集作为约束补充。本文最终建议以综合时间模型为主方案，并将影响力 Top20 方案作为宣传和承办能力约束下的备选。

人口加权成本的定义：对球队 i 到场地 k 的人口加权成本为 $c_{ik}^{\text{pop}} = P_i \cdot t_{ik}$，其中 P_i 为球队 i 所在地的常住人口（万人），t_{ik} 为公路旅行时间（分钟）。该定义的含义是：将人口较多地区的球迷出行时间赋予更高权重，使选址倾向于服务人口密集区域。

5.4 选址的等效成本扩展

上述多维实验采用“替换成本矩阵或限制候选集”的方式扩展基准模型。若赛事组织方希望把更多软目标放入同一个目标函数，也可在本节基础上使用如下等效成本形式：

$$c_{ik}^{\text{ext}} = \alpha d_{ik} + \beta h_{ik} - \gamma I_k + \delta r_k + \eta b_{ik}. \tag{24}$$

其中 d_{ik} 可取距离或旅行时间，h_{ik} 为主场惩罚，I_k 为城市影响力得分，r_k 为区域失衡惩罚，b_{ik} 为山区、海岛或跨湾交通不便修正项。本文当前实证只显式使用距离/时间成本、人口加权成本和影响力 Top20 候选集；其余项作为赛事组织细化时的扩展接口。具体含义如下：

(1) **主场公平**：本文已将主场禁止作为硬约束；若实际组织中允许有限主场因素，也可改为惩罚项 h_{ik}。

(2) **城市影响力**：优先选择地级市和经济强县作为场地（I_k 项），或直接限制候选集为人口与收入综合得分最高的 Top20 地点，以扩大赛事传播效果。

(3) **区域覆盖**：通过 r_k 项惩罚区域集中，确保浙北、浙中、浙南和沿海地区均有场地分布。

6 问题 4：赛制合理化建议

6.1 仿真模型

6.1.1 Bradley-Terry 模型

设球队 i 的实力为 s_i，则球队 i 战胜球队 j 的概率为：

$$P(i \text{ 胜 } j) = \frac{s_i}{s_i + s_j} \tag{25}$$

6.1.2 同分排序与进球数扰动

主仿真聚焦赛制公平性，因此单场只判胜负，不引入平局。真实足球小组赛采用积分 3/1/0 制，平局是常见的比赛结果；当前“必分胜负”的简化会高估单场比赛的排序信息量。本文将含平局积分制作为后续扩展，正文只比较同一简化模型下单循环和双循环的相对差异。小组赛积分相同时，用进球数作为第二排序指标；为避免完全确定性的并列排序，令球队 i 对阵球队 j 时的进球贡献近似服从 Poisson 分布：

$$G_{ij} \sim \text{Poisson}(\lambda_{ij}), \quad \lambda_{ij} = 0.5 + 0.2\, s_i - 0.1\, s_j. \tag{26}$$

进球期望 λ_{ij} 同时取决于攻方实力 s_i 和守方实力 s_j：强队对弱队时 λ 更高，强队对强队时 λ 更低，符合足球比赛的攻守对抗规律。在本题所有实力取值下 $\lambda_{ij} \geq 0.1$，保证 Poisson 参数始终为正。该 Poisson 项只用于同分时的进球数比较，不改变 Bradley-Terry 模型给出的胜负概率。

6.1.3 实力赋值

前文采用行政层级的三档固定赋值来估算实力，但固定三档赋值会导致同档球队完全并列，因此在这里的仿真采用层内随机化实力模型。每次完整赛事仿真前，按球队层级抽取潜在真实实力：

$$s_i = \max\{\varepsilon,\ \mu_{\ell(i)} + \xi_i\}, \quad \xi_i \sim N(0, \sigma^2), \quad \sigma = \frac{1}{3}, \tag{27}$$

其中 $\mu_{\ell(i)} \in \{3, 2, 1\}$ 分别对应市级、县级市和县队，$\varepsilon = 0.05$ 用于保证 Bradley-Terry 概率中的实力值为正。相邻层级均值差为 3σ，既保留层级强弱差异，也允许同档球队之间存在合理波动，使每次仿真中各队实力各不相同。

6.2 评价指标

(1) **Spearman 相关系数**：最终排名与真实实力排名的相关程度，衡量赛制对实力排序的保真度。由于淘汰赛只产生部分顺序，本文按比赛可观察信息补全名次：冠军列第 1 名，随后按淘汰轮次由后到前排列同轮出局球队；同轮出局球队不再用真实实力排序，而按当轮随机对阵顺序补位。小组赛未出线的 32 队按小组第 3 名、第 4 名分层列为 33–64 名，同层随机补位。不同补全规则会影响 Spearman 系数的绝对值，因此这里只在同一补位规则下比较赛制差异。

(2) **晋级准确率**：真实实力 Top-32 球队中实际晋级 32 强的比例。

(3) **冠军集中度**：统计实力 Top-1、Top-4、Top-8 球队的夺冠概率。

6.3 仿真结果

对当前赛制（单循环小组赛+单场淘汰赛）和替代赛制（双循环小组赛+单场淘汰赛）各进行 5 000 次完整赛事仿真，结果如表 9。

表 9: 赛制方案对比（层内正态随机实力模型，$\sigma = 1/3$，5 000 次模拟）

指标	单循环+淘汰（当前）	双循环+淘汰（替代）
Spearman 相关系数（均值）	0.471	**0.564**
Top-32 晋级准确率	69.15%	**74.59%**
Top-1 夺冠率	7.58%	7.66%
Top-4 夺冠率	26.76%	27.44%
Top-8 夺冠率	47.12%	48.74%
Spearman 5%分位	0.293	0.409
Spearman 95%分位	0.630	0.702
每组比赛场次	6	12
总比赛场次	$16 \times 6 + 31 = 127$	$16 \times 12 + 31 = 223$

双循环显著改善小组出线和整体排名保真度，但对冠军集中度的提升有限。其原因是冠军仍由 32 强单场淘汰产生，淘汰赛偶然性成为冠军归属的主要扰动来源。

6.4 分析与建议

6.4.1 当前赛制的优势与不足

优势：

(1) 赛程紧凑（127 场比赛），组织成本低。

(2) 淘汰赛单场制悬念大、观赏性高。

(3) 16 组同时开赛，初期关注面广。

不足：

(1) Spearman 相关系数仅 0.471，最终名次与实力排序只有中等程度一致，赛制对真实实力的保真度仍有提升空间。

(2) Top-32 晋级准确率仅约 69.2%，约三成实力前 32 强球队会因小组赛偶然性无缘淘汰赛。

(3) 单循环每队仅 3 场比赛，样本量小导致小组排名偶然性大。

6.4.2 改进建议

表 9 只对”小组赛单循环/双循环”这一核心变量进行了可复现实证比较。以下建议中，”小组赛改为双循环”有直接仿真支撑；其余建议属于基于仿真诊断和赛事组织逻辑提出的结构性增强方案，可在后续版本中继续通过完整赛程仿真验证。

建议 1：小组赛改为双循环：Spearman 相关系数从 0.471 提升至 0.564，Top-32 晋级准确率从 69.2%提升至 74.6%。代价是比赛场次增加 96 场（约 75.6%），需权衡赛事周期和成本。

建议 2：引入种子分档抽签：在问题 2 抽签流程的基础上增加分档约束。可按历史成绩、行政层级和上届成绩将 64 支队伍分为 4 档，每档 16 支，抽签时每组各取一档球队，同时继续保留 C1、C2、C3 地域约束。实现上可将“每组每档 1 队”加入前瞻流的剩余容量状态，先筛除会破坏分档容量或地域约束的候选组，再进行现场抽取。该机制有助于减少极端“死亡之组”，提高小组赛和淘汰赛阶段的竞技均衡。

建议 3：采用同组回避的交叉对阵：淘汰赛首轮采用“小组第 1 名 vs 其他组第 2 名”的交叉规则，并保证同组球队首轮不相遇；若采用种子排名，还可避免综合实力排名靠前的队伍过早相遇。这能在不显著增加场次的情况下提升淘汰赛后半程的比赛质量。

建议 4：淘汰赛关键轮次采用两回合制：随机实力模型中，双循环对 Top-8 夺冠率的影响有限（表 9），说明冠军偶然性主要来自单场淘汰。从八强或四强阶段引入主客场两回合制可进一步降低关键轮次偶然性。该建议属于赛制结构外推，完整的定量验证需在仿真中显式加入两回合总比分、加时或点球规则，本文不直接给出两回合晋级概率。

建议 5：增设小组第三附加赛与排位赛：16 个小组第三名可进行单败附加赛，决出前 4 名并设置“优胜杯”或专项奖励，但不影响冠军争夺。该设计能让中游球队在小组赛后仍有明确目标，并提高县级队和基层球迷的参与感。小组第四名及未进入附加赛的队伍也可安排短程排位赛或交流赛，扩大赛事覆盖面。

建议 6：双败淘汰作为试点增强方案：若赛事周期和组织能力允许，可在 32 强阶段试点双败淘汰赛。胜者组负者进入败者组，败者组优胜者与胜者组冠军会师决赛；若胜者组冠军在总决赛失利，可设置一次加赛。双败机制能减少“一场失误即出局”的偶然性，但赛程、转播和观众理解成本均高，建议先作为试点或在八强以后局部采用。

建议 7：动态平衡赛程：根据问题 3 得到的旅行距离和旅行时间矩阵，合理安排比赛间隔，避免某队短期内连续长途旅行。双循环、附加赛或双败赛制若被采用，应同步做赛程整数规划，使场地利用、休息时间和出行负担保持均衡。

6.4.3 分阶段实施路径

综合公平性收益和组织成本，建议采用分阶段落地方案。第一阶段优先实施双循环小组赛，这是当前仿真直接支持、且规则解释成本较低的改动；同组回避交叉对阵可作为低成本配套规则同步试行。第二阶段引入种子分档抽签和小组第三附加赛，用较小的额外组织成本改善小组均衡和参赛体验。第三阶段再试点双败淘汰或关键轮次两回合制，用于进一步提高冠军归属与真实实力的一致性。

7 结论

本文围绕浙江省市县联赛的赛事组织问题，建立了从分组、抽签、选址到赛制评价的完整数学模型与算法框架。主要贡献如下：

(1) **分组方案**：建立了 CSP 模型，提出贪心、蛇形轮转和 ILP 四种分组策略。ILP 均衡方案实现了 $F_1 = 0$、$F_2 = 0.484$、$F_2' = 1$ 的最优均衡性能。

(2) **抽签方案**：设计了前瞻最小费用流辅助抽签法，将 C1、C2 硬约束与 C3 最小冲突目标统一到网络流前向校验中。该方案在每次抽出县级队后只开放能够延续全局最优剩余分配的小组集合，既避免现场反悔换组，也在配对实验中实现零失败和 $F_1 = 0$。

(3) **比赛地点**：建立了带主场禁止约束的设施选址+容量指派 ILP 模型，在 haversine 距离成本下得到总旅行距离 7 801 km 的最优方案。多维成本实验表明，交通时间、人口加权和影响力候选集会改变具体场地集合，但诸暨、杭州、义乌、绍兴等高频场地具有较强稳健性。

(4) **赛制建议**：通过 5 000 次赛事仿真，定量证明双循环小组赛可将 Spearman 相关系数从 0.471 提升至 0.564，Top-32 晋级准确率从 69.2%提升至 74.6%。结合冠军集中度结果，本文进一步建议引入种子分档、同组回避交叉对阵、小组第三附加赛、关键轮次两回合制和双败淘汰试点。

(5) **抽签鲁棒性验证**：在问题 2 中通过 100 个随机行政拓扑比较基础贪心与前瞻网络流，发现前瞻流在全部配对实验中均未死锁；当 C3 零冲突不可实现时，网络流模型会自动转为最小冲突分配，进一步支持采用前瞻流进行事前筛选。

本文建立的模型框架可以为类似层级结构体育赛事的分组与赛制设计提供方法论参考。

附录 A：分组方案详细结果

方案 A（贪心优先填充）

组 1 [实力=7] 温州市(市)，遂昌县(县)，浦江县(县)，余姚市(县市级)
组 2 [实力=8] 丽水市(市)，平阳县(县)，东阳市(县市级)，江山市(县市级)
组 3 [实力=7] 金华市(市)，永嘉县(县)，玉环市(县市级)，常山县(县)
组 4 [实力=7] 台州市(市)，文成县(县)，桐乡市(县市级)，龙游县(县)
组 5 [实力=6] 嘉兴市(市)，景宁县(县)，仙居县(县)，开化县(县)
组 6 [实力=8] 宁波市(市)，龙泉市(县市级)，天台县(县)，建德市(县市级)
组 7 [实力=7] 衢州市(市)，松阳县(县)，温岭市(县市级)，桐庐县(县)
组 8 [实力=6] 杭州市(市)，云和县(县)，三门县(县)，安吉县(县)
组 9 [实力=7] 湖州市(市)，缙云县(县)，临海市(县市级)，淳安县(县)
组10 [实力=6] 绍兴市(市)，青田县(县)，嘉善县(县)，德清县(县)
组11 [实力=7] 舟山市(市)，庆元县(县)，海宁市(县市级)，长兴县(县)
组12 [实力=6] 乐清市(县市级)，兰溪市(县市级)，海盐县(县)，新昌县(县)
组13 [实力=6] 苍南县(县)，磐安县(县)，平湖市(县市级)，诸暨市(县市级)
组14 [实力=8] 瑞安市(县市级)，永康市(县市级)，慈溪市(县市级)，嵊州市(县市级)
组15 [实力=5] 泰顺县(县)，义乌市(县市级)，象山县(县)，岱山县(县)
组16 [实力=5] 龙港市(县市级)，武义县(县)，宁海县(县)，嵊泗县(县)

指标：F1=0 F2=0.927 F2'=3

方案 B（蛇形轮转分配）

组 1 [实力=8] 温州市(市)，东阳市(县市级)，玉环市(县市级)，长兴县(县)
组 2 [实力=7] 丽水市(市)，乐清市(县市级)，浦江县(县)，仙居县(县)
组 3 [实力=6] 金华市(市)，苍南县(县)，天台县(县)，德清县(县)
组 4 [实力=7] 台州市(市)，瑞安市(县市级)，武义县(县)，安吉县(县)
组 5 [实力=8] 嘉兴市(市)，泰顺县(县)，义乌市(县市级)，温岭市(县市级)
组 6 [实力=8] 宁波市(市)，龙港市(县市级)，永康市(县市级)，三门县(县)
组 7 [实力=7] 衢州市(市)，平阳县(县)，磐安县(县)，临海市(县市级)
组 8 [实力=8] 杭州市(市)，永嘉县(县)，兰溪市(县市级)，桐乡市(县市级)
组 9 [实力=6] 湖州市(市)，文成县(县)，庆元县(县)，嘉善县(县)
组10 [实力=7] 绍兴市(市)，遂昌县(县)，海宁市(县市级)，淳安县(县)
组11 [实力=6] 舟山市(市)，景宁县(县)，海盐县(县)，桐庐县(县)
组12 [实力=7] 龙泉市(县市级)，平湖市(县市级)，建德市(县市级)，新昌县(县)
组13 [实力=6] 松阳县(县)，慈溪市(县市级)，开化县(县)，诸暨市(县市级)
组14 [实力=5] 云和县(县)，象山县(县)，龙游县(县)，嵊州市(县市级)
组15 [实力=4] 缙云县(县)，宁海县(县)，常山县(县)，岱山县(县)

组16 [实力=6] 青田县(县), 余姚市(县市级), 江山市(县市级), 嵊泗县(县)

指标: F1=0 F2=1.111 F2'=4

方案 C（ILP-C3 最小化）

组 1 [实力=8] 宁波市(市), 平湖市(县市级), 东阳市(县市级), 天台县(县)
组 2 [实力=6] 象山县(县), 泰顺县(县), 金华市(市), 仙居县(县)
组 3 [实力=5] 永嘉县(县), 嵊州市(县市级), 龙游县(县), 松阳县(县)
组 4 [实力=6] 平阳县(县), 长兴县(县), 台州市(市), 庆元县(县)
组 5 [实力=5] 慈溪市(县市级), 嘉善县(县), 新昌县(县), 嵊泗县(县)
组 6 [实力=6] 宁海县(县), 桐乡市(县市级), 德清县(县), 江山市(县市级)
组 7 [实力=6] 桐庐县(县), 苍南县(县), 磐安县(县), 丽水市(市)
组 8 [实力=8] 建德市(县市级), 舟山市(市), 临海市(县市级), 遂昌县(县)
组 9 [实力=8] 瑞安市(县市级), 嘉兴市(市), 义乌市(县市级), 开化县(县)
组10 [实力=6] 乐清市(县市级), 安吉县(县), 常山县(县), 玉环市(县市级)
组11 [实力=8] 龙港市(县市级), 湖州市(市), 永康市(县市级), 云和县(县)
组12 [实力=6] 淳安县(县), 温州市(市), 武义县(县), 景宁县(县)
组13 [实力=7] 文成县(县), 诸暨市(县市级), 兰溪市(县市级), 龙泉市(县市级)
组14 [实力=7] 余姚市(县市级), 海盐县(县), 衢州市(市), 青田县(县)
组15 [实力=7] 海宁市(县市级), 绍兴市(市), 三门县(县), 缙云县(县)
组16 [实力=7] 杭州市(市), 浦江县(县), 岱山县(县), 温岭市(县市级)

指标: F1=0 F2=0.992 F2'=3

方案 D（ILP 均衡优化）—— 推荐方案

组 1 [实力=6] 龙港市(县市级), 德清县(县), 东阳市(县市级), 三门县(县)
组 2 [实力=6] 嘉善县(县), 金华市(市), 岱山县(县), 庆元县(县)
组 3 [实力=7] 永嘉县(县), 平湖市(县市级), 长兴县(县), 舟山市(市)
组 4 [实力=7] 温州市(市), 海盐县(县), 诸暨市(县市级), 磐安县(县)
组 5 [实力=7] 余姚市(县市级), 安吉县(县), 浦江县(县), 丽水市(市)
组 6 [实力=7] 杭州市(市), 象山县(县), 兰溪市(县市级), 常山县(县)
组 7 [实力=7] 宁波市(市), 江山市(县市级), 仙居县(县), 青田县(县)
组 8 [实力=6] 建德市(县市级), 苍南县(县), 龙游县(县), 温岭市(县市级)
组 9 [实力=7] 武义县(县), 衢州市(市), 玉环市(县市级), 云和县(县)
组10 [实力=7] 文成县(县), 绍兴市(市), 永康市(县市级), 开化县(县)
组11 [实力=7] 宁海县(县), 嘉兴市(市), 义乌市(县市级), 遂昌县(县)
组12 [实力=6] 瑞安市(县市级), 海宁市(县市级), 新昌县(县), 松阳县(县)
组13 [实力=6] 桐庐县(县), 乐清市(县市级), 临海市(县市级), 景宁县(县)

组14 [实力=7] 淳安县(县), 慈溪市(县市级), 台州市(市), 缙云县(县)
组15 [实力=7] 平阳县(县), 桐乡市(县市级), 嵊州市(县市级), 龙泉市(县市级)
组16 [实力=6] 泰顺县(县), 湖州市(市), 嵊泗县(县), 天台县(县)

指标: F1=0 F2=0.484 F2'=1

ILP 最优选址方案（基于方案 D 分组）

场地1: 余姚市 承办 组2、组3 场地2: 新昌县 承办 组12、组16
场地3: 东阳市 承办 组1、组11 场地4: 永康市 承办 组10、组15
场地5: 武义县 承办 组8、组9 场地6: 浦江县 承办 组5、组6
场地7: 磐安县 承办 组4、组14 场地8: 仙居县 承办 组7、组13

总旅行距离: 7801 km | 平均: 121.9 km/队 | 最远: 337 km

附录 B：代码清单

所有代码使用 Python 3 实现，依赖 NumPy、SciPy 和 PuLP（调用 COIN-OR CBC 求解器）。

B.1 问题 1：分组方案

```python
#!/usr/bin/env python3
"""
Q1: 浙超分组方案设计与比较
==============================
- 方案A: 贪心优先填充（启发式）
- 方案B: 蛇形轮转分配（启发式）
- 方案C: 整数规划最优（ILP）
- 方案D: ILP兼顾实力均衡（多目标）

评价指标: F1(C3违反数) F2(实力标准差) F3(来源多样性熵)
"""

import numpy as np
from itertools import combinations
from collections import defaultdict
from dataclasses import dataclass

# ============================================================
# 1. DATA
# ============================================================
```

```
NUM_GROUPS = 16
GROUP_SIZE = 4
STRENGTH_MAP = {"municipal": 3, "city": 2, "county": 1}

CITY_DATA = {
    "杭州": {
        "municipal": "杭州市",
        "county_teams": [
            ("建德市", "city"),
            ("桐庐县", "county"),
            ("淳安县", "county"),
        ],
    },
    "宁波": {
        "municipal": "宁波市",
        "county_teams": [
            ("余姚市", "city"),
            ("慈溪市", "city"),
            ("象山县", "county"),
            ("宁海县", "county"),
        ],
    },
    "温州": {
        "municipal": "温州市",
        "county_teams": [
            ("瑞安市", "city"),
            ("乐清市", "city"),
            ("龙港市", "city"),
            ("永嘉县", "county"),
            ("平阳县", "county"),
            ("苍南县", "county"),
            ("文成县", "county"),
            ("泰顺县", "county"),
        ],
    },
    "嘉兴": {
        "municipal": "嘉兴市",
        "county_teams": [
            ("海宁市", "city"),
            ("平湖市", "city"),
            ("桐乡市", "city"),
            ("嘉善县", "county"),
            ("海盐县", "county"),
        ],
```

```
    },
    "湖州": {
        "municipal": "湖州市",
        "county_teams": [
            ("德清县", "county"),
            ("长兴县", "county"),
            ("安吉县", "county"),
        ],
    },
    "绍兴": {
        "municipal": "绍兴市",
        "county_teams": [
            ("诸暨市", "city"),
            ("嵊州市", "city"),
            ("新昌县", "county"),
        ],
    },
    "金华": {
        "municipal": "金华市",
        "county_teams": [
            ("兰溪市", "city"),
            ("义乌市", "city"),
            ("东阳市", "city"),
            ("永康市", "city"),
            ("武义县", "county"),
            ("浦江县", "county"),
            ("磐安县", "county"),
        ],
    },
    "衢州": {
        "municipal": "衢州市",
        "county_teams": [
            ("江山市", "city"),
            ("常山县", "county"),
            ("开化县", "county"),
            ("龙游县", "county"),
        ],
    },
    "舟山": {
        "municipal": "舟山市",
        "county_teams": [
            ("岱山县", "county"),
            ("嵊泗县", "county"),
        ],
    },
```

```python
    "台州": {
        "municipal": "台州市",
        "county_teams": [
            ("温岭市", "city"),
            ("临海市", "city"),
            ("玉环市", "city"),
            ("三门县", "county"),
            ("天台县", "county"),
            ("仙居县", "county"),
        ],
    },
    "丽水": {
        "municipal": "丽水市",
        "county_teams": [
            ("龙泉市", "city"),
            ("青田县", "county"),
            ("缙云县", "county"),
            ("遂昌县", "county"),
            ("松阳县", "county"),
            ("云和县", "county"),
            ("庆元县", "county"),
            ("景宁县", "county"),
        ],
    },
}


@dataclass
class Team:
    name: str
    city: str
    level: str  # municipal / city / county
    strength: int


def build_teams():
    teams = []
    for city, data in CITY_DATA.items():
        teams.append(Team(data["municipal"], city, "municipal", 3))
        for name, level in data["county_teams"]:
            teams.append(Team(name, city, level, STRENGTH_MAP[level]))
    return teams


TEAMS = build_teams()
```

```python
TEAM_INDEX = {t.name: i for i, t in enumerate(TEAMS)}

LEVEL_TAG = {"municipal": "市", "city": "县市级", "county": "县"}


# ============================================================
# 2. CONSTRAINT CHECKING
# ============================================================


def check_c1(groups):
    """C1: 各市级队分配在不同小组"""
    violations = []
    for j, group in enumerate(groups):
        munis = [n for n in group if TEAMS[TEAM_INDEX[n]].level == "municipal"]
        if len(munis) > 1:
            violations.append((j + 1, munis))
    return violations


def check_c2(groups):
    """C2: 市级队不与同市县级队同组"""
    violations = []
    for j, group in enumerate(groups):
        members = [TEAMS[TEAM_INDEX[n]] for n in group]
        for t in members:
            if t.level == "municipal":
                same_city = [o.name for o in members if o.city == t.city and o.name != t.name]
                if same_city:
                    violations.append((j + 1, t.name, same_city))
    return violations


def check_c3(groups):
    """C3: 同市县级队尽量不在同一组 → 返回 (违反对数, 详情)"""
    total = 0
    details = []
    for j, group in enumerate(groups):
        members = [TEAMS[TEAM_INDEX[n]] for n in group]
        county = [t for t in members if t.level != "municipal"]
        by_city = defaultdict(list)
        for t in county:
            by_city[t.city].append(t.name)
```

```python
        for city, names in by_city.items():
            if len(names) > 1:
                pairs = len(names) * (len(names) - 1) // 2
                total += pairs
                details.append((j + 1, city, names))
    return total, details


# ============================================================
# 3. EVALUATION METRICS
# ============================================================


def metric_f1(groups):
    """F1: 同市县级队同组的配对数 (C(n,2) per group per city)"""
    total = 0
    for group in groups:
        members = [TEAMS[TEAM_INDEX[n]] for n in group]
        county = [t for t in members if t.level != "municipal"]
        by_city = defaultdict(int)
        for t in county:
            by_city[t.city] += 1
        for c in by_city.values():
            total += c * (c - 1) // 2
    return total


def metric_f2(groups):
    """F2: 各组实力总和的标准差"""
    strengths = [sum(TEAMS[TEAM_INDEX[n]].strength for n in g) for g in
        groups]
    return float(np.std(strengths))


def metric_f2_range(groups):
    """F2': 各组实力极差"""
    strengths = [sum(TEAMS[TEAM_INDEX[n]].strength for n in g) for g in
        groups]
    return max(strengths) - min(strengths)


def metric_f3(groups):
    """F3: 组内城市来源多样性的平均 Shannon 熵"""
    entropies = []
    for group in groups:
```

```python
        members = [TEAMS[TEAM_INDEX[n]] for n in group]
        by_city = defaultdict(int)
        for t in members:
            by_city[t.city] += 1
        total = len(group)
        h = -sum((c / total) * np.log(c / total) for c in by_city.values()
            if c > 0)
        entropies.append(h)
    return float(np.mean(entropies))


def evaluate(groups, label):
    """返回指标字典并打印详情"""
    c1v = check_c1(groups)
    c2v = check_c2(groups)
    c3n, c3d = check_c3(groups)

    f1 = metric_f1(groups)
    f2 = metric_f2(groups)
    f2r = metric_f2_range(groups)
    f3 = metric_f3(groups)

    strengths = [sum(TEAMS[TEAM_INDEX[n]].strength for n in g) for g in
        groups]

    print(f"\n{'=' * 72}")
    print(f"  {label}")
    print(f"{'=' * 72}")

    for j, group in enumerate(groups):
        parts = []
        for n in group:
            t = TEAMS[TEAM_INDEX[n]]
            parts.append(f"{n}({LEVEL_TAG[t.level]})")
        print(f"  组{j + 1:2d} [实力={strengths[j]:2d}] {', '.join(parts)}"
            )

    print(f"\n  约束检查:")
    print(f"    C1: {'通过' if not c1v else '违反 ' + str(c1v)}")
    print(f"    C2: {'通过' if not c2v else '违反 ' + str(c2v)}")
    print(f"    C3: {'通过' if c3n == 0 else f'{c3n} 对冲突'}")
    for g, city, names in c3d:
        print(f"        组{g} {city}: {names}")

    print(f"\n  评价指标:")
```

```python
    print(f"    F1  C3冲突对数 (↓) : {f1}")
    print(f"    F2  实力标准差  (↓) : {f2:.4f}")
    print(f"    F2' 实力极差    (↓) : {f2r}")
    print(f"    F3  多样性熵    (↑) : {f3:.4f}")

    return {"label": label, "F1": f1, "F2": f2, "F2_range": f2r, "F3": f3}


# ============================================================
# 4. SCHEME A — GREEDY PRIORITY FILL
# ============================================================


def scheme_a_greedy(seed=42):
    """按县级队数量从多到少处理各市，每支县级队优先放入容量最充裕且不违反约束的组"""
    rng = np.random.RandomState(seed)
    groups = [[] for _ in range(NUM_GROUPS)]

    # 市级队 → 组 1-11
    cities_desc = sorted(CITY_DATA, key=lambda c: len(CITY_DATA[c]["county_teams"]), reverse=True)
    mun_group = {}
    for i, city in enumerate(cities_desc):
        groups[i].append(CITY_DATA[city]["municipal"])
        mun_group[city] = i

    # 县级队贪心填充
    for city in cities_desc:
        forbidden = mun_group[city]
        teams = list(CITY_DATA[city]["county_teams"])
        rng.shuffle(teams)
        used_groups = set()  # 本市已用组 (C3)

        for name, _level in teams:
            candidates = []
            for j in range(NUM_GROUPS):
                if j == forbidden or len(groups[j]) >= GROUP_SIZE:
                    continue
                # C3: 该组已有本市县级队则降优先
                has_same = any(
                    TEAMS[TEAM_INDEX[n]].city == city and TEAMS[TEAM_INDEX[n]].level != "municipal"
                    for n in groups[j]
                )
```

```python
                candidates.append((j, has_same, len(groups[j])))

            # 优先: 无C3冲突 → 人最少
            candidates.sort(key=lambda x: (x[1], x[2]))
            best = candidates[0][0]
            groups[best].append(name)

    return groups


# ============================================================
# 5. SCHEME B — SERPENTINE ROUND-ROBIN
# ============================================================


def scheme_b_serpentine(seed=42):
    """蛇形轮转: 市级队入组后, 县级队按蛇形序号依次分配到可用组"""
    rng = np.random.RandomState(seed)
    groups = [[] for _ in range(NUM_GROUPS)]

    cities_desc = sorted(CITY_DATA, key=lambda c: len(CITY_DATA[c]["county_teams"]), reverse=True)
    mun_group = {}
    for i, city in enumerate(cities_desc):
        groups[i].append(CITY_DATA[city]["municipal"])
        mun_group[city] = i

    # 全局蛇形序列 (1,2,...,16,16,...,2,1,1,2,...)
    serp = list(range(NUM_GROUPS))
    for _ in range(10):
        serp += serp[-NUM_GROUPS:][::-1]

    pos = 0  # 蛇形指针
    for city in cities_desc:
        forbidden = mun_group[city]
        teams = list(CITY_DATA[city]["county_teams"])
        rng.shuffle(teams)
        used = set()

        for name, _level in teams:
            placed = False
            # 沿蛇形序列找可用组
            for _ in range(NUM_GROUPS):
                g = serp[pos % len(serp)]
                pos += 1
```

```
                if g == forbidden or len(groups[g]) >= GROUP_SIZE:
                    continue
                if g in used:
                    continue  # C3
                groups[g].append(name)
                used.add(g)
                placed = True
                break

            if not placed:
                # 放松 C3
                for g in range(NUM_GROUPS):
                    if g != forbidden and len(groups[g]) < GROUP_SIZE and g not in used:
                        groups[g].append(name)
                        used.add(g)
                        placed = True
                        break
                if not placed:
                    for g in range(NUM_GROUPS):
                        if g != forbidden and len(groups[g]) < GROUP_SIZE:
                            groups[g].append(name)
                            placed = True
                            break

    return groups


# ============================================================
# 6. SCHEME C — ILP (MINIMIZE C3 VIOLATIONS)
# ============================================================


def scheme_c_ilp():
    """整数线性规划: 硬约束 C1+C2, 目标最小化 C3 冲突对数"""
    from pulp import LpMinimize, LpProblem, LpVariable, lpSum, LpBinary, LpStatus, value, PULP_CBC_CMD

    N = len(TEAMS)
    prob = LpProblem("ZheChao_C3opt", LpMinimize)

    x = {(i, j): LpVariable(f"x_{i}_{j}", cat=LpBinary) for i in range(N) for j in range(NUM_GROUPS)}

    # 同市县级队配对
```

```python
    pairs = []
    for city, data in CITY_DATA.items():
        idxs = [TEAM_INDEX[n] for n, _ in data["county_teams"]]
        for a, b in combinations(idxs, 2):
            pairs.append((a, b))

    y = {(p, j): LpVariable(f"y_{p}_{j}", cat=LpBinary) for p in range(len(
        pairs)) for j in range(NUM_GROUPS)}

    # --- 约束 ---
    # 每支球队恰在一组
    for i in range(N):
        prob += lpSum(x[i, j] for j in range(NUM_GROUPS)) == 1

    # 每组恰好4队
    for j in range(NUM_GROUPS):
        prob += lpSum(x[i, j] for i in range(N)) == GROUP_SIZE

    # C1: 每组至多1支市级队
    mun_idx = [i for i, t in enumerate(TEAMS) if t.level == "municipal"]
    for j in range(NUM_GROUPS):
        prob += lpSum(x[i, j] for i in mun_idx) <= 1

    # C2: 市级队与同市县级队不同组
    for city, data in CITY_DATA.items():
        mi = TEAM_INDEX[data["municipal"]]
        for name, _ in data["county_teams"]:
            ki = TEAM_INDEX[name]
            for j in range(NUM_GROUPS):
                prob += x[mi, j] + x[ki, j] <= 1

    # y 链接
    for p, (a, b) in enumerate(pairs):
        for j in range(NUM_GROUPS):
            prob += y[p, j] >= x[a, j] + x[b, j] - 1

    # 目标: 最小化 C3 冲突
    prob += lpSum(y[p, j] for p in range(len(pairs)) for j in range(
        NUM_GROUPS))

    prob.solve(PULP_CBC_CMD(msg=0, timeLimit=120))

    if LpStatus[prob.status] != "Optimal":
        print(f"  [ILP] 状态: {LpStatus[prob.status]}")
        return None
```

```python

    groups = [[] for _ in range(NUM_GROUPS)]
    for i in range(N):
        for j in range(NUM_GROUPS):
            if value(x[i, j]) > 0.5:
                groups[j].append(TEAMS[i].name)
                break
    return groups


# ============================================================
# 7. SCHEME D — ILP + STRENGTH BALANCE (MULTI-OBJECTIVE)
# ============================================================


def scheme_d_ilp_balanced(w_c3=10, w_str=1):
    """ILP多目标: min w_c3·C3违反 + w_str·(S_max - S_min)"""
    from pulp import LpMinimize, LpProblem, LpVariable, lpSum, LpBinary, LpStatus, value, PULP_CBC_CMD

    N = len(TEAMS)
    prob = LpProblem("ZheChao_Balanced", LpMinimize)

    x = {(i, j): LpVariable(f"x_{i}_{j}", cat=LpBinary) for i in range(N) for j in range(NUM_GROUPS)}

    pairs = []
    for city, data in CITY_DATA.items():
        idxs = [TEAM_INDEX[n] for n, _ in data["county_teams"]]
        for a, b in combinations(idxs, 2):
            pairs.append((a, b))

    y = {(p, j): LpVariable(f"y_{p}_{j}", cat=LpBinary) for p in range(len(pairs)) for j in range(NUM_GROUPS)}

    s_max = LpVariable("S_max")
    s_min = LpVariable("S_min")

    # 约束 (同 Scheme C)
    for i in range(N):
        prob += lpSum(x[i, j] for j in range(NUM_GROUPS)) == 1
    for j in range(NUM_GROUPS):
        prob += lpSum(x[i, j] for i in range(N)) == GROUP_SIZE
    mun_idx = [i for i, t in enumerate(TEAMS) if t.level == "municipal"]
    for j in range(NUM_GROUPS):
```

```python
        prob += lpSum(x[i, j] for i in mun_idx) <= 1
    for city, data in CITY_DATA.items():
        mi = TEAM_INDEX[data["municipal"]]
        for name, _ in data["county_teams"]:
            ki = TEAM_INDEX[name]
            for j in range(NUM_GROUPS):
                prob += x[mi, j] + x[ki, j] <= 1
    for p, (a, b) in enumerate(pairs):
        for j in range(NUM_GROUPS):
            prob += y[p, j] >= x[a, j] + x[b, j] - 1

    # 实力范围约束
    for j in range(NUM_GROUPS):
        gs = lpSum(TEAMS[i].strength * x[i, j] for i in range(N))
        prob += s_max >= gs
        prob += s_min <= gs

    prob += w_c3 * lpSum(y[p, j] for p in range(len(pairs)) for j in range(
        NUM_GROUPS)) + w_str * (s_max - s_min)

    prob.solve(PULP_CBC_CMD(msg=0, timeLimit=120))

    if LpStatus[prob.status] != "Optimal":
        print(f"  [ILP] 状态: {LpStatus[prob.status]}")
        return None

    groups = [[] for _ in range(NUM_GROUPS)]
    for i in range(N):
        for j in range(NUM_GROUPS):
            if value(x[i, j]) > 0.5:
                groups[j].append(TEAMS[i].name)
                break
    return groups


# ============================================================
# 8. COMPARISON TABLE
# ============================================================


def print_comparison(results):
    print(f"\n{'=' * 72}")
    print(f"  方案对比汇总")
    print(f"{'=' * 72}")
```

```python
    header = f"  {'方案':<28} {'F1(C3,↓)':<10} {'F2( ,↓)':<10} {'F2(极差,↓)':<12} {'F3(熵,↑)':<10}"
    print(header)
    print(f"  {'-' * 70}")
    for r in results:
        print(
            f"  {r['label']:<28} {r['F1']:<10} "
            f"{r['F2']:<10.4f} {r['F2_range']:<12} {r['F3']:<10.4f}"
        )

    # 排名 (各指标分别排名, 然后综合)
    print(f"\n  各指标排名 (1=最优):")
    for key, direction, nice in [("F1", 1, "C3违反"), ("F2", 1, "实力 "), ("F3", -1, "多样性熵")]:
        sorted_r = sorted(results, key=lambda r: r[key] * direction)
        print(f"    {nice}: ", end="")
        print(" > ".join(f"{r['label'][:6]}({i + 1})" for i, r in enumerate(sorted_r)))


# ============================================================
# MAIN
# ============================================================


def main():
    n_mun = sum(1 for t in TEAMS if t.level == "municipal")
    n_cc = sum(1 for t in TEAMS if t.level == "city")
    n_co = sum(1 for t in TEAMS if t.level == "county")
    total_strength = sum(t.strength for t in TEAMS)

    print("=" * 72)
    print("  浙超分组方案 — 问题1: 分组方案设计与比较")
    print(f"  共 {len(TEAMS)} 支球队 | {n_mun} 市级 + {n_cc} 县级市 + {n_co} 县")
    print(f"  {NUM_GROUPS} 组 × {GROUP_SIZE} 队 | 总实力 {total_strength} | 组均 {total_strength / NUM_GROUPS:.2f}")
    print("=" * 72)

    results = []

    groups_a = scheme_a_greedy(seed=42)
    results.append(evaluate(groups_a, "方案A: 贪心优先填充"))

    groups_b = scheme_b_serpentine(seed=42)
```

```python
    results.append(evaluate(groups_b, "方案B: 蛇形轮转分配"))

    groups_c = scheme_c_ilp()
    if groups_c is not None:
        results.append(evaluate(groups_c, "方案C: ILP(C3最小)"))

    groups_d = scheme_d_ilp_balanced(w_c3=10, w_str=1)
    if groups_d is not None:
        results.append(evaluate(groups_d, "方案D: ILP(均衡)"))

    print_comparison(results)


if __name__ == "__main__":
    main()
```

Listing 1: q1_grouping.py —— 球队数据、约束检查、四种分组方案（贪心、蛇形轮转、ILP-C3 最小化、ILP 均衡优化）及 F_1, F_2, F_2' 三项评价指标

B.2 问题 2：前瞻抽签与鲁棒性检验

```python
#!/usr/bin/env python3
"""
Q2: Computer-assisted look-ahead draw by min-cost flow.

This script implements the draw mechanism discussed in the paper draft:

1. Draw municipal teams first and place them into distinct groups.
2. Draw county-level teams in a fully random order.
3. For the drawn county team, test each currently legal group.
4. Keep only groups that still allow the remaining teams to be completed with
   minimum possible C3 conflict count.

The forward check is a min-cost max-flow model. C1 and C2 are hard constraints.
C3 is encoded as a marginal cost: putting the next county team of city c into
group j costs the number of already placed county teams of city c in group j.
Thus the accumulated cost is exactly the additional C3 pair count.
"""

from __future__ import annotations
```

```python

import argparse
from collections import deque
from dataclasses import dataclass
from typing import Dict, Iterable, List, Optional, Tuple

import numpy as np

from q1_grouping import (
    CITY_DATA,
    GROUP_SIZE,
    LEVEL_TAG,
    NUM_GROUPS,
    TEAM_INDEX,
    TEAMS,
    check_c1,
    check_c2,
    check_c3,
    metric_f1,
    metric_f2,
    metric_f2_range,
    metric_f3,
)


# ============================================================
# 1. Min-cost max-flow
# ============================================================


@dataclass
class _Edge:
    to: int
    rev: int
    cap: int
    cost: int


class MinCostMaxFlow:
    """Small integer min-cost max-flow by successive shortest augmenting
        path."""

    def __init__(self, n: int):
        self.graph: List[List[_Edge]] = [[] for _ in range(n)]

```

```python
    def add_edge(self, u: int, v: int, cap: int, cost: int) -> None:
        if cap <= 0:
            return
        fwd = _Edge(v, len(self.graph[v]), cap, cost)
        rev = _Edge(u, len(self.graph[u]), 0, -cost)
        self.graph[u].append(fwd)
        self.graph[v].append(rev)

    def flow(self, s: int, t: int, target: int) -> Tuple[int, int]:
        n = len(self.graph)
        sent = 0
        total_cost = 0

        while sent < target:
            dist = [10**18] * n
            parent: List[Optional[Tuple[int, int]]] = [None] * n
            in_queue = [False] * n

            dist[s] = 0
            queue = deque([s])
            in_queue[s] = True

            while queue:
                u = queue.popleft()
                in_queue[u] = False

                for ei, edge in enumerate(self.graph[u]):
                    if edge.cap <= 0:
                        continue
                    nd = dist[u] + edge.cost
                    if nd < dist[edge.to]:
                        dist[edge.to] = nd
                        parent[edge.to] = (u, ei)
                        if not in_queue[edge.to]:
                            queue.append(edge.to)
                            in_queue[edge.to] = True

            if parent[t] is None:
                break

            add = target - sent
            v = t
            while v != s:
                u, ei = parent[v]  # type: ignore[misc]
                add = min(add, self.graph[u][ei].cap)
```

```
                v = u

            v = t
            while v != s:
                u, ei = parent[v]  # type: ignore[misc]
                edge = self.graph[u][ei]
                edge.cap -= add
                self.graph[v][edge.rev].cap += add
                total_cost += add * edge.cost
                v = u

            sent += add

        return sent, total_cost


@dataclass
class _FlowEdge:
    to: int
    rev: int
    cap: int


class Dinic:
    """Max-flow used as a fast F1=0 feasibility test."""

    def __init__(self, n: int):
        self.graph: List[List[_FlowEdge]] = [[] for _ in range(n)]

    def add_edge(self, u: int, v: int, cap: int) -> None:
        if cap <= 0:
            return
        fwd = _FlowEdge(v, len(self.graph[v]), cap)
        rev = _FlowEdge(u, len(self.graph[u]), 0)
        self.graph[u].append(fwd)
        self.graph[v].append(rev)

    def max_flow(self, s: int, t: int) -> int:
        flow = 0
        n = len(self.graph)

        while True:
            level = [-1] * n
            level[s] = 0
            queue = deque([s])
```

```python

        while queue:
            u = queue.popleft()
            for edge in self.graph[u]:
                if edge.cap > 0 and level[edge.to] < 0:
                    level[edge.to] = level[u] + 1
                    queue.append(edge.to)

        if level[t] < 0:
            return flow

        it = [0] * n

        def dfs(u: int, pushed: int) -> int:
            if u == t:
                return pushed
            while it[u] < len(self.graph[u]):
                ei = it[u]
                edge = self.graph[u][ei]
                if edge.cap > 0 and level[u] + 1 == level[edge.to]:
                    ret = dfs(edge.to, min(pushed, edge.cap))
                    if ret:
                        edge.cap -= ret
                        self.graph[edge.to][edge.rev].cap += ret
                        return ret
                it[u] += 1
            return 0

        while True:
            pushed = dfs(s, 10**9)
            if not pushed:
                break
            flow += pushed


# ============================================================
# 2. Flow model for the remaining draw
# ============================================================


def group_label(j: int) -> str:
    return f"G{j + 1:02d}"


```

```python
def county_counts_by_city_group(groups: List[List[str]]) -> Dict[str, List[int]]:
    counts = {city: [0] * NUM_GROUPS for city in CITY_DATA}
    for j, group in enumerate(groups):
        for name in group:
            team = TEAMS[TEAM_INDEX[name]]
            if team.level != "municipal":
                counts[team.city][j] += 1
    return counts


def c3_from_counts(counts: Dict[str, List[int]]) -> int:
    return sum(n * (n - 1) // 2 for row in counts.values() for n in row)


def remaining_capacity(groups: List[List[str]]) -> List[int]:
    return [GROUP_SIZE - len(group) for group in groups]


def future_min_c3_cost(
    groups: List[List[str]],
    mun_group: Dict[str, int],
    remaining_by_city: Dict[str, int],
) -> Optional[int]:
    """
    Return the minimum additional C3 pair count needed to place all remaining
    county-level teams. None means the current partial draw is infeasible.
    """

    need = sum(remaining_by_city.values())
    if need == 0:
        return 0

    residual = remaining_capacity(groups)
    if sum(residual) != need:
        return None

    active_cities = [city for city, n in remaining_by_city.items() if n > 0]
    counts = county_counts_by_city_group(groups)

    source = 0
    city_base = 1
    group_base = city_base + len(active_cities)
```

```python
    sink = group_base + NUM_GROUPS
    mcmf = MinCostMaxFlow(sink + 1)

    for ci, city in enumerate(active_cities):
        city_node = city_base + ci
        demand = remaining_by_city[city]
        mcmf.add_edge(source, city_node, demand, 0)

        for j in range(NUM_GROUPS):
            if j == mun_group[city] or residual[j] <= 0:
                continue

            # Convex C3 penalty, linearized by parallel unit-capacity edges:
            # extra 1st team costs a_cj, extra 2nd costs a_cj + 1, ...
            max_units = min(demand, residual[j])
            base_cost = counts[city][j]
            for k in range(max_units):
                mcmf.add_edge(city_node, group_base + j, 1, base_cost + k)

    for j, cap in enumerate(residual):
        mcmf.add_edge(group_base + j, sink, cap, 0)

    sent, cost = mcmf.flow(source, sink, need)
    if sent != need:
        return None
    return cost


def future_zero_c3_feasible(
    groups: List[List[str]],
    mun_group: Dict[str, int],
    remaining_by_city: Dict[str, int],
) -> bool:
    """
    Fast check for whether all remaining teams can still be placed with F1=0.

    This is a max-flow model with city-group capacity 1 after excluding groups
    where that city already has a county-level team.
    """

    need = sum(remaining_by_city.values())
    if need == 0:
```

```python
        return True

    residual = remaining_capacity(groups)
    if sum(residual) != need:
        return False

    counts = county_counts_by_city_group(groups)
    if c3_from_counts(counts) > 0:
        return False

    active_cities = [city for city, n in remaining_by_city.items() if n > 0]
    source = 0
    city_base = 1
    group_base = city_base + len(active_cities)
    sink = group_base + NUM_GROUPS
    dinic = Dinic(sink + 1)

    for ci, city in enumerate(active_cities):
        city_node = city_base + ci
        dinic.add_edge(source, city_node, remaining_by_city[city])

        for j in range(NUM_GROUPS):
            if j == mun_group[city] or residual[j] <= 0:
                continue
            if counts[city][j] == 0:
                dinic.add_edge(city_node, group_base + j, 1)

    for j, cap in enumerate(residual):
        dinic.add_edge(group_base + j, sink, cap)

    return dinic.max_flow(source, sink) == need


def candidate_final_f1(
    groups: List[List[str]],
    mun_group: Dict[str, int],
    remaining_by_city: Dict[str, int],
    team_name: str,
    group_id: int,
    *,
    hard_zero_first: bool = True,
) -> Optional[Tuple[int, bool]]:
    """Return (best final F1, used_min_cost_fallback) for this candidate group."""
```

```python

    team = TEAMS[TEAM_INDEX[team_name]]
    if team.level == "municipal":
        raise ValueError("candidate_final_f1 is only for county-level teams")
    if group_id == mun_group[team.city] or len(groups[group_id]) >= GROUP_SIZE:
        return None

    groups[group_id].append(team_name)

    if hard_zero_first and future_zero_c3_feasible(groups, mun_group, remaining_by_city):
        groups[group_id].pop()
        return 0, False

    current_f1 = c3_from_counts(county_counts_by_city_group(groups))
    future_cost = future_min_c3_cost(groups, mun_group, remaining_by_city)
    groups[group_id].pop()

    if future_cost is None:
        return None
    return current_f1 + future_cost, True


# ============================================================
# 3. Look-ahead draw
# ============================================================


@dataclass
class DrawDecision:
    step: int
    team: str
    city: str
    legal_count: int
    feasible_count: int
    safe_count: int
    chosen_group: int
    best_final_f1: int


@dataclass
class DrawStats:
    flow_checks: int = 0
```

```python
    min_cost_checks: int = 0
    feasible_candidates: int = 0
    safe_candidates: int = 0
    unsafe_candidates: int = 0
    infeasible_candidates: int = 0
    dominated_candidates: int = 0
    critical_steps: int = 0
    infeasible_steps: int = 0
    dominated_steps: int = 0
    forced_steps: int = 0
    max_best_final_f1: int = 0
    decisions: List[DrawDecision] = None  # type: ignore[assignment]

    def __post_init__(self) -> None:
        if self.decisions is None:
            self.decisions = []


def all_county_teams() -> List[str]:
    teams = []
    for data in CITY_DATA.values():
        teams.extend(name for name, _level in data["county_teams"])
    return teams


def initial_remaining_by_city() -> Dict[str, int]:
    return {city: len(data["county_teams"]) for city, data in CITY_DATA.
        items()}


def draw_lookahead_flow(
    rng: np.random.Generator,
    *,
    trace: bool = False,
) -> Tuple[List[List[str]], DrawStats]:
    """
    Run one physical-style draw.

    Municipal teams are drawn first. County-level teams are then drawn from
        one
    mixed pool in random order. For each drawn county-level team, the group
        pot
    contains only groups that preserve minimum attainable final F1.
    """

```

```python
    groups: List[List[str]] = [[] for _ in range(NUM_GROUPS)]
    stats = DrawStats()

    # Stage 1: municipal teams.
    cities = list(CITY_DATA.keys())
    rng.shuffle(cities)
    group_slots = np.arange(NUM_GROUPS)
    rng.shuffle(group_slots)

    mun_group: Dict[str, int] = {}
    for i, city in enumerate(cities):
        g = int(group_slots[i])
        groups[g].append(CITY_DATA[city]["municipal"])
        mun_group[city] = g

    # Stage 2: county-level teams in a random physical order.
    county_pool = all_county_teams()
    rng.shuffle(county_pool)
    remaining_by_city = initial_remaining_by_city()

    for step, team_name in enumerate(county_pool, start=1):
        team = TEAMS[TEAM_INDEX[team_name]]
        remaining_by_city[team.city] -= 1

        legal_groups = [
            j
            for j in range(NUM_GROUPS)
            if len(groups[j]) < GROUP_SIZE and j != mun_group[team.city]
        ]

        scores: List[Tuple[int, int]] = []
        for j in legal_groups:
            result = candidate_final_f1(
                groups, mun_group, remaining_by_city, team_name, j
            )
            stats.flow_checks += 1
            if result is not None:
                score, used_min_cost = result
                if used_min_cost:
                    stats.min_cost_checks += 1
                scores.append((j, score))

        if not scores:
            raise RuntimeError(f"No feasible look-ahead group for {team_name}")
```

```
        best_final_f1 = min(score for _j, score in scores)
        safe_groups = [j for j, score in scores if score == best_final_f1]
        feasible_count = len(scores)
        safe_count = len(safe_groups)
        infeasible_count = len(legal_groups) - feasible_count
        dominated_count = feasible_count - safe_count
        unsafe_count = len(legal_groups) - safe_count

        stats.feasible_candidates += feasible_count
        stats.safe_candidates += safe_count
        stats.unsafe_candidates += unsafe_count
        stats.infeasible_candidates += infeasible_count
        stats.dominated_candidates += dominated_count
        if unsafe_count > 0:
            stats.critical_steps += 1
        if infeasible_count > 0:
            stats.infeasible_steps += 1
        if dominated_count > 0:
            stats.dominated_steps += 1

        chosen = int(rng.choice(safe_groups))
        groups[chosen].append(team_name)

        if safe_count == 1:
            stats.forced_steps += 1
        stats.max_best_final_f1 = max(stats.max_best_final_f1,
            best_final_f1)

        if trace:
            stats.decisions.append(
                DrawDecision(
                    step=step,
                    team=team_name,
                    city=team.city,
                    legal_count=len(legal_groups),
                    feasible_count=feasible_count,
                    safe_count=safe_count,
                    chosen_group=chosen,
                    best_final_f1=best_final_f1,
                )
            )

    return groups, stats
```

```python

# ============================================================
# 4. Reporting
# ============================================================


def print_groups(groups: List[List[str]]) -> None:
    strengths = [sum(TEAMS[TEAM_INDEX[name]].strength for name in g) for g in groups]
    for j, group in enumerate(groups):
        parts = []
        for name in group:
            team = TEAMS[TEAM_INDEX[name]]
            parts.append(f"{name}({LEVEL_TAG[team.level]})")
        print(f"  {group_label(j)} [strength={strengths[j]:2d}] " + ", ".join(parts))


def print_evaluation(groups: List[List[str]], stats: DrawStats) -> None:
    c1v = check_c1(groups)
    c2v = check_c2(groups)
    c3n, c3d = check_c3(groups)
    print()
    print(
        "  约束检查: "
        f"C1={'通过' if not c1v else '违反'}  "
        f"C2={'通过' if not c2v else '违反'}  "
        f"C3={'通过' if c3n == 0 else str(c3n) + ' 对冲突'}"
    )
    if c3d:
        for g, city, names in c3d:
            print(f"    C3详情: {group_label(g - 1)} {city} {names}")

    print(
        "  评价指标: "
        f"F1={metric_f1(groups)}  "
        f"F2={metric_f2(groups):.4f}  "
        f"F2'={metric_f2_range(groups)}  "
        f"F3={metric_f3(groups):.4f}"
    )
    print(
        "  前瞻统计: "
        f"流检查次数={stats.flow_checks}  "
        f"最小费用流回退={stats.min_cost_checks}  "
        f"危险候选组={stats.unsafe_candidates}  "
```

```
        f"干预步数={stats.critical_steps}  "
        f"唯一选择步数={stats.forced_steps}  "
        f"最大最终F1={stats.max_best_final_f1}"
    )


def print_trace(stats: DrawStats, limit: int) -> None:
    if limit <= 0:
        return

    print()
    print(f"  前 {min(limit, len(stats.decisions))} 步县级队抽签详情:")
    print("  步序  队伍        城市  可选  可行  安全  选定组  最优最终F1")
    print("  ----  ----------  ----  ----  ----  ----  ------
        -------------")
    for d in stats.decisions[:limit]:
        print(
            f"  {d.step:>4}  {d.team:<10}  {d.city:<4}  "
            f"{d.legal_count:>5}  {d.feasible_count:>4}  {d.safe_count:>4}
                "
            f"{group_label(d.chosen_group):>6}  {d.best_final_f1:>13}"
        )


def run_sample(seed: int, trace_limit: int) -> None:
    rng = np.random.default_rng(seed)
    groups, stats = draw_lookahead_flow(rng, trace=trace_limit > 0)

    print("=" * 72)
    print(f"前瞻最小费用流抽签示例, seed={seed}")
    print("=" * 72)
    print_groups(groups)
    print_evaluation(groups, stats)
    print_trace(stats, trace_limit)


def summarize(values: Iterable[float]) -> Tuple[float, float, float, float
    ]:
    arr = np.array(list(values), dtype=float)
    return float(arr.mean()), float(np.median(arr)), float(arr.min()),
        float(arr.max())


def run_monte_carlo(n_sim: int, seed0: int) -> None:
    f1_values: List[int] = []
```

```
    f2_values: List[float] = []
    f2r_values: List[int] = []
    f3_values: List[float] = []
    flow_checks: List[int] = []
    min_cost_checks: List[int] = []
    unsafe_candidates: List[int] = []
    critical_steps: List[int] = []
    forced_steps: List[int] = []
    failures = 0

    for k in range(n_sim):
        rng = np.random.default_rng(seed0 + k)
        try:
            groups, stats = draw_lookahead_flow(rng)
        except Exception:
            failures += 1
            continue

        f1_values.append(metric_f1(groups))
        f2_values.append(metric_f2(groups))
        f2r_values.append(metric_f2_range(groups))
        f3_values.append(metric_f3(groups))
        flow_checks.append(stats.flow_checks)
        min_cost_checks.append(stats.min_cost_checks)
        unsafe_candidates.append(stats.unsafe_candidates)
        critical_steps.append(stats.critical_steps)
        forced_steps.append(stats.forced_steps)

    print()
    print("=" * 72)
    print(f"蒙特卡洛汇总, n={n_sim}, seed0={seed0}, 失败次数={failures}")
    print("=" * 72)

    if not f1_values:
        print("无成功抽签样本。")
        return

    f1 = np.array(f1_values)
    f2 = np.array(f2_values)
    f2r = np.array(f2r_values)
    f3 = np.array(f3_values)
    fc = np.array(flow_checks)
    mc = np.array(min_cost_checks)
    unsafe = np.array(unsafe_candidates)
    critical = np.array(critical_steps)
```

```
    fs = np.array(forced_steps)

    print(
        f"  F1 C3冲突对数: 均值={f1.mean():.4f}, "
        f"P(F1=0)={(f1 == 0).mean():.4f}, 最大={int(f1.max())}"
    )
    print(
        f"  F2 实力标准差: 均值={f2.mean():.4f}, "
        f"中位={np.median(f2):.4f}, 最小={f2.min():.4f}, 最大={f2.max():.4f}"
    )
    print(
        f"  F2' 实力极差:   均值={f2r.mean():.2f}, "
        f"最小={int(f2r.min())}, 最大={int(f2r.max())}"
    )
    print(
        f"  F3 多样性熵:    均值={f3.mean():.4f}, "
        f"最小={f3.min():.4f}, 最大={f3.max():.4f}"
    )
    print(
        f"  前瞻流工作量:   平均流检查={fc.mean():.1f}, "
        f"平均最小费用流回退={mc.mean():.1f}, "
        f"平均唯一选择步数={fs.mean():.1f}"
    )
    print(
        f"  前瞻流剪枝:     平均危险候选组={unsafe.mean():.1f}, "
        f"危险率={(unsafe.sum() / fc.sum()):.4f}, "
        f"平均干预步数={critical.mean():.1f}"
    )


def main() -> None:
    parser = argparse.ArgumentParser(
        description="Q2 look-ahead draw with min-cost max-flow forward checking."
    )
    parser.add_argument("--seed", type=int, default=42, help="sample draw seed")
    parser.add_argument("--sim", type=int, default=50, help="number of MC runs")
    parser.add_argument("--seed0", type=int, default=1000, help="first MC seed")
    parser.add_argument(
        "--trace",
        type=int,
```

```
        default=12,
        help="how many county draw decisions to print for the sample",
    )
    args = parser.parse_args()

    run_sample(args.seed, args.trace)
    if args.sim > 0:
        run_monte_carlo(args.sim, args.seed0)


if __name__ == "__main__":
    main()
```

Listing 2: q2_lookahead_flow.py —— Dinic 最大流与最小费用最大流实现，前瞻辅助抽签法，含 C3 零冲突快速最大流检查

```
"""
策略对比实验: 修复(repair) vs 重启(restart) vs 前瞻流(look-ahead flow)
==============================================================

实验设计:
  - 用同一批随机种子驱动四种策略:
    1. greedy_no_repair: 贪心, 死锁即失败
    2. greedy_with_repair: 贪心 + _repair_swap 修复
    3. restart: 贪心, 死锁或C3>0则重抽直到C3=0 (模拟队友方案)
    4. lookahead_flow: 随机抽队 + 前瞻网络流筛选安全小组
  - 对比各策略在 F1/F2/F2'/F3 上的条件分布和期望

理论框架:
  - 二部图模型: 县级队    可用组槽位
  - Hall 婚配定理保证完美匹配存在
  - 修复/重启/前瞻流对应解空间上的不同采样分布
"""

import sys, os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import numpy as np
import time
import argparse
from q1_grouping import (
    CITY_DATA,
    TEAMS,
    TEAM_INDEX,
```

```
    NUM_GROUPS,
    GROUP_SIZE,
    metric_f1,
    metric_f2,
    metric_f3,
)
from q2_lookahead_flow import draw_lookahead_flow


def greedy_algorithm(rng, repair=False):
    """贪心 + 修复，理论上永远不失败"""
    groups = [[] for _ in range(NUM_GROUPS)]
    flag = False  # 是否已经修复过一次了

    cities_shuffled = list(CITY_DATA.keys())
    rng.shuffle(cities_shuffled)
    slots = np.arange(NUM_GROUPS)
    rng.shuffle(slots)

    mun_group = {}
    for i, city in enumerate(cities_shuffled):
        g = int(slots[i])
        groups[g].append(CITY_DATA[city]["municipal"])
        mun_group[city] = g

    cities_desc = sorted(
        CITY_DATA.keys(),
        key=lambda c: len(CITY_DATA[c]["county_teams"]),
        reverse=True,
    )

    for city in cities_desc:
        forbidden = mun_group[city]
        teams = list(CITY_DATA[city]["county_teams"])
        rng.shuffle(teams)

        for name, _level in teams:
            candidates = []
            for j in range(NUM_GROUPS):
                if j == forbidden or len(groups[j]) >= GROUP_SIZE:
                    continue
                has_c3 = any(
                    TEAMS[TEAM_INDEX[n]].city == city
                    and TEAMS[TEAM_INDEX[n]].level != "municipal"
                    for n in groups[j]
```

```python
                )
                candidates.append((j, has_c3, len(groups[j])))

            if not candidates and repair:
                repaired = _repair_swap(groups, city, forbidden, mun_group)
                flag = True
                if repaired is not None:
                    j_swap, freed_j = repaired
                    groups[freed_j].append(name)
                    continue
                raise RuntimeError(f"Dead end (even after repair): {name} from {city}")
            elif not candidates:
                return None, flag

            candidates.sort(key=lambda x: (x[1], x[2]))
            best_size = candidates[0][2]
            best_c3 = candidates[0][1]
            top = [j for j, c3, sz in candidates if c3 == best_c3 and sz == best_size]
            g = int(rng.choice(top))
            groups[g].append(name)

    return groups, flag


# ============================================================
# 策略1: 贪心无修复 (基线)
# ============================================================


def greedy_no_repair(rng):
    """贪心, 死锁返回 None"""
    return greedy_algorithm(rng, repair=False)[0]


# ============================================================
# 策略2: 贪心 + 修复 (我们的方案)
# ============================================================


def _repair_swap(groups, city, forbidden, mun_group):
    if len(groups[forbidden]) >= GROUP_SIZE:
        return None
    for j in range(NUM_GROUPS):
```

```python
        if j == forbidden or len(groups[j]) < GROUP_SIZE:
            continue
        for idx, other_name in enumerate(groups[j]):
            other = TEAMS[TEAM_INDEX[other_name]]
            if other.city == city or other.level == "municipal":
                continue
            other_forbidden = mun_group[other.city]
            if other_forbidden == forbidden:
                continue
            groups[j].pop(idx)
            groups[forbidden].append(other_name)
            return (j, j)
    return None


def greedy_with_repair(rng):
    """贪心 + 修复，理论上永远不失败"""
    try:
        return greedy_algorithm(rng, repair=True)
    except RuntimeError:
        return None, False


# ============================================================
# 策略3: 重启 (队友方案模拟)
# ============================================================


def restart_until_good(rng, max_retries=50):
    """
    反复贪心重抽，直到 C3=0 或超过重试上限.
    用同一 rng 的不同子状态模拟独立重抽.
    返回 (groups, retries_used).
    """
    for attempt in range(max_retries):
        # 每次重试用 rng 衍生一个新种子，确保独立性
        child_seed = rng.integers(0, 2**31)
        child_rng = np.random.default_rng(child_seed)

        groups = greedy_no_repair(child_rng)
        if groups is None:
            continue  # 死锁，重试
        if metric_f1(groups) == 0:
            return groups, attempt + 1
    return None, max_retries  # 未找到
```

```python


# ============================================================
# 策略4: 前瞻最小费用流
# ============================================================


def lookahead_flow_strategy(rng):
    """
    随机抽县级队，每次用网络流前瞻筛选不会破坏全局最小C3的安全小组.
    返回 (groups, stats).
    """
    return draw_lookahead_flow(rng, trace=False)


# ============================================================
# 实验主体
# ============================================================


def run_experiment(n_seeds=100, include_marginal=True):
    print("=" * 72)
    print("  策略对比实验: 贪心 / 修复 / 重启 / 前瞻流")
    print(f"  {n_seeds} 个随机种子")
    print("=" * 72)

    results = {
        "no_repair": {
            "f1": [],
            "f2": [],
            "f2r": [],
            "f3": [],
            "n": 0,
            "deadlocks": 0,
            "total_time": 0,
        },
        "with_repair": {
            "f1": [],
            "f2": [],
            "f2r": [],
            "f3": [],
            "n": 0,
            "deadlocks": 0,
            "repaired": 0,
            "fixed_deadlocks": 0,
```

```
            "total_time": 0,
        },
        "restart": {
            "f1": [],
            "f2": [],
            "f2r": [],
            "f3": [],
            "n": 0,
            "deadlocks": 0,
            "not_found": 0,
            "retries": [],
            "total_time": 0,
        },
        "lookahead_flow": {
            "f1": [],
            "f2": [],
            "f2r": [],
            "f3": [],
            "n": 0,
            "failures": 0,
            "flow_checks": [],
            "min_cost_checks": [],
            "unsafe_candidates": [],
            "critical_steps": [],
            "forced_steps": [],
            "total_time": 0,
        },
    }

    # 边际均匀性计数器（三个策略各一个）
    team_names = list(TEAM_INDEX.keys())
    n_teams = len(team_names)
    marginal_counts = {
        "no_repair": np.zeros((n_teams, NUM_GROUPS), dtype=int),
        "with_repair": np.zeros((n_teams, NUM_GROUPS), dtype=int),
        "restart": np.zeros((n_teams, NUM_GROUPS), dtype=int),
        "lookahead_flow": np.zeros((n_teams, NUM_GROUPS), dtype=int),
    }
    marginal_n_success = {
        "no_repair": 0,
        "with_repair": 0,
        "restart": 0,
        "lookahead_flow": 0,
    }

```

```
    for sd in range(n_seeds):
        # seed = random.randint(0, 2**31 - 1)  # 生成一个随机种子
        seed = sd  # 使用固定种子以确保可复现
        rng = np.random.default_rng(seed)

        # --- 策略1: 无修复 ---
        t1 = time.perf_counter()
        g1 = greedy_no_repair(rng)
        t2 = time.perf_counter()
        results["no_repair"]["total_time"] += t2 - t1
        if g1 is None:
            results["no_repair"]["deadlocks"] += 1
        else:
            results["no_repair"]["n"] += 1
            marginal_n_success["no_repair"] += 1
            results["no_repair"]["f1"].append(metric_f1(g1))
            strengths = [sum(TEAMS[TEAM_INDEX[n]].strength for n in grp)
                for grp in g1]
            results["no_repair"]["f2"].append(float(np.std(strengths)))
            results["no_repair"]["f2r"].append(max(strengths) - min(
                strengths))
            results["no_repair"]["f3"].append(metric_f3(g1))
            for g_idx, group in enumerate(g1):
                for name in group:
                    marginal_counts["no_repair"][TEAM_INDEX[name], g_idx]
                        += 1

        # --- 策略2: 有修复 ---
        rng2 = np.random.default_rng(seed)  # 同一种子
        t1 = time.perf_counter()
        g2, flag = greedy_with_repair(rng2)
        t2 = time.perf_counter()
        results["with_repair"]["total_time"] += t2 - t1
        if g2 is not None:
            results["with_repair"]["n"] += 1
            marginal_n_success["with_repair"] += 1
            f1_r = metric_f1(g2)
            results["with_repair"]["f1"].append(f1_r)
            strengths = [sum(TEAMS[TEAM_INDEX[n]].strength for n in grp)
                for grp in g2]
            results["with_repair"]["f2"].append(float(np.std(strengths)))
            results["with_repair"]["f2r"].append(max(strengths) - min(
                strengths))
            results["with_repair"]["f3"].append(metric_f3(g2))
            for g_idx, group in enumerate(g2):
```

```
                for name in group:
                    marginal_counts["with_repair"][TEAM_INDEX[name], g_idx] += 1
            # 检查是否经过了修复 (与无修复结果不同)
            results["with_repair"]["fixed_deadlocks"] += int(flag)
            if f1_r > 0:
                results["with_repair"]["repaired"] += 1

        # --- 策略3: 重启 ---
        rng3 = np.random.default_rng(seed)
        t1 = time.perf_counter()
        g3, retries = restart_until_good(rng3)
        t2 = time.perf_counter()
        results["restart"]["total_time"] += t2 - t1
        if g3 is None:
            results["restart"]["not_found"] += 1
        else:
            results["restart"]["n"] += 1
            marginal_n_success["restart"] += 1
            results["restart"]["f1"].append(metric_f1(g3))
            strengths = [sum(TEAMS[TEAM_INDEX[n]].strength for n in grp) for grp in g3]
            results["restart"]["f2"].append(float(np.std(strengths)))
            results["restart"]["f2r"].append(max(strengths) - min(strengths))
            results["restart"]["f3"].append(metric_f3(g3))
            results["restart"]["retries"].append(retries)
            for g_idx, group in enumerate(g3):
                for name in group:
                    marginal_counts["restart"][TEAM_INDEX[name], g_idx] += 1

        # --- 策略4: 前瞻最小费用流 ---
        rng4 = np.random.default_rng(seed)
        t1 = time.perf_counter()
        try:
            g4, look_stats = lookahead_flow_strategy(rng4)
        except Exception:
            g4, look_stats = None, None
        t2 = time.perf_counter()
        results["lookahead_flow"]["total_time"] += t2 - t1
        if g4 is None:
            results["lookahead_flow"]["failures"] += 1
        else:
            results["lookahead_flow"]["n"] += 1
```

```
            marginal_n_success["lookahead_flow"] += 1
            results["lookahead_flow"]["f1"].append(metric_f1(g4))
            strengths = [sum(TEAMS[TEAM_INDEX[n]].strength for n in grp) for grp in g4]
            results["lookahead_flow"]["f2"].append(float(np.std(strengths)))
            results["lookahead_flow"]["f2r"].append(max(strengths) - min(strengths))
            results["lookahead_flow"]["f3"].append(metric_f3(g4))
            results["lookahead_flow"]["flow_checks"].append(look_stats.flow_checks)
            results["lookahead_flow"]["min_cost_checks"].append(look_stats.min_cost_checks)
            results["lookahead_flow"]["unsafe_candidates"].append(look_stats.unsafe_candidates)
            results["lookahead_flow"]["critical_steps"].append(look_stats.critical_steps)
            results["lookahead_flow"]["forced_steps"].append(look_stats.forced_steps)
            for g_idx, group in enumerate(g4):
                for name in group:
                    marginal_counts["lookahead_flow"][TEAM_INDEX[name], g_idx] += 1

    # ============================================================
    # 输出结果
    # ============================================================

    for name, label in [
        ("no_repair", "策略1: 贪心无修复"),
        ("with_repair", "策略2: 贪心+修复"),
        ("restart", "策略3: 重启至C3=0"),
        ("lookahead_flow", "策略4: 前瞻最小费用流"),
    ]:
        r = results[name]
        print(f"\n{'-' * 72}")
        print(f"  {label}")
        print(f"{'-' * 72}")
        print(f"  成功次数: {r['n']}/{n_seeds}")
        print(f"  总时间: {r['total_time']:.4f} 秒")
        if name == "no_repair":
            print(f"  死锁次数: {r['deadlocks']} ({r['deadlocks'] / n_seeds:.2%})")
        if name == "with_repair":
            print(f"  修复后C3>0的次数: {r['repaired']}")
```

```
            print(f"  修复了死锁的次数: {r['fixed_deadlocks']}")
        if name == "restart":
            print(f"  未找到C3=0的次数: {r['not_found']}")
            if r["retries"]:
                print(f"  平均重试次数: {np.mean(r['retries']):.2f}")
        if name == "lookahead_flow":
            print(f"  失败次数: {r['failures']} ({r['failures'] / n_seeds:.2%})")
            if r["flow_checks"]:
                print(f"  平均前瞻流检查次数: {np.mean(r['flow_checks']):.1f}")
                print(f"  平均最小费用流回退次数: {np.mean(r['min_cost_checks']):.1f}")
                unsafe_rate = np.sum(r["unsafe_candidates"]) / np.sum(r["flow_checks"])
                print(f"  平均危险候选组数: {np.mean(r['unsafe_candidates']):.1f}")
                print(f"  危险候选组占比: {unsafe_rate:.2%}")
                print(f"  平均需要前瞻干预步骤数: {np.mean(r['critical_steps']):.1f}")
                print(f"  平均唯一安全组步骤数: {np.mean(r['forced_steps']):.1f}")

        if r["f1"]:
            arr_f1 = np.array(r["f1"])
            arr_f2 = np.array(r["f2"])
            arr_f2r = np.array(r["f2r"])
            arr_f3 = np.array(r["f3"])

            print(f"\n  F1 (C3冲突对数, down=good):")
            print(
                f"    均值={np.mean(arr_f1):.4f}  P(F1=0)={np.mean(arr_f1 == 0):.4f}  max={int(np.max(arr_f1))}"
            )

            print(f"  F2 (实力标准差, down=good):")
            print(f"    均值={np.mean(arr_f2):.4f}  中位={np.median(arr_f2):.4f}")
            print(
                f"    P(F2<=0.5)={np.mean(arr_f2 <= 0.5):.4f}  P(F2<=1.0)={np.mean(arr_f2 <= 1.0):.4f}"
            )

            print(f"  F2'(实力极差, down=good):")
            print(
```

```
                f"    均值={np.mean(arr_f2r):.2f}  P(range<=2)={np.mean(arr_f2r <= 2):.4f}"
            )

            print(f"  F3 (多样性熵, up=good):")
            print(f"    均值={np.mean(arr_f3):.4f}")

    # ============================================================
    # 关键对比: 同一种子下 repair vs restart 的 F2/F3 差异
    # ============================================================
    print(f"\n{'=' * 72}")
    print(f"  核心问题: 修复是否系统性劣化 F2/F3?")
    print(f"{'=' * 72}")

    # 找出 repair 的 F1>0 的那些 case (即修复引入了 C3 违反)
    repair_f1 = np.array(results["with_repair"]["f1"])
    repair_f2 = np.array(results["with_repair"]["f2"])
    restart_f2 = np.array(results["restart"]["f2"])
    lookahead_f1 = np.array(results["lookahead_flow"]["f1"])
    lookahead_f2 = np.array(results["lookahead_flow"]["f2"])

    f1_zero_mask = repair_f1 == 0
    f1_pos_mask = repair_f1 > 0

    n_zero = int(f1_zero_mask.sum())
    n_pos = int(f1_pos_mask.sum())

    print(f"\n  修复策略中:")
    print(f"    C3=0 的次数: {n_zero} ({n_zero / len(repair_f1):.2%})")
    print(f"    C3>0 的次数: {n_pos} ({n_pos / len(repair_f1):.2%})")

    if n_pos > 0:
        print(
            f"\n    C3>0 (被修复过) 时的 F2 均值: {repair_f2[f1_pos_mask].mean():.4f}"
        )
        print(f"    C3=0 (未被修复) 时的 F2 均值: {repair_f2[f1_zero_mask].mean():.4f}")
        print(f"    重启策略 (始终C3=0)  的 F2 均值: {restart_f2.mean():.4f}")

    print(f"\n  全样本 F2 均值对比:")
    print(f"    修复策略: {repair_f2.mean():.4f}")
    print(f"    重启策略: {restart_f2.mean():.4f}")
    if len(lookahead_f2) > 0:
```

```
        print(f"    前瞻流策略: {lookahead_f2.mean():.4f}")

    repair_f3 = np.array(results["with_repair"]["f3"])
    restart_f3 = np.array(results["restart"]["f3"])
    lookahead_f3 = np.array(results["lookahead_flow"]["f3"])

    print(f"\n  全样本 F3 均值对比:")
    print(f"    修复策略: {repair_f3.mean():.4f}")
    print(f"    重启策略: {restart_f3.mean():.4f}")
    if len(lookahead_f3) > 0:
        print(f"    前瞻流策略: {lookahead_f3.mean():.4f}")

    if len(lookahead_f1) > 0:
        print(f"\n  前瞻流 C3 严格性:")
        print(
            f"    P(F1=0): {np.mean(lookahead_f1 == 0):.4f}, "
            f"max F1: {int(np.max(lookahead_f1))}"
        )

    if not include_marginal:
        return results

    # ============================================================
    # 边际均匀性检验 (复用上方循环中的 marginal_counts)
    # ============================================================
    from scipy.stats import chisquare, kstest

    strategies = [
        ("no_repair", "S1: greedy (no repair)"),
        ("with_repair", "S2: greedy + repair"),
        ("restart", "S3: restart until C3=0"),
        ("lookahead_flow", "S4: look-ahead min-cost flow"),
    ]

    for skey, slabel in strategies:
        ns = marginal_n_success[skey]
        if ns == 0:
            continue
        expected = ns / NUM_GROUPS
        mres = []
        for tname in team_names:
            i = TEAM_INDEX[tname]
            team = TEAMS[i]
            stat, p_val = chisquare(marginal_counts[skey][i], f_exp=
                expected)
```

```
            mres.append(
                {
                    "name": tname,
                    "city": team.city,
                    "level": team.level,
                    "chi2": stat,
                    "p_value": p_val,
                    "pass": p_val >= 0.05,
                }
            )

        n_pass = sum(r["pass"] for r in mres)
        n_total = len(mres)
        alpha_bonf = 0.05 / n_total
        n_pass_bonf = sum(r["p_value"] >= alpha_bonf for r in mres)
        all_p = [r["p_value"] for r in mres]

        print(f"\n{'=' * 72}")
        print(f"  边际均匀性检验 - {slabel}")
        print(f"  {ns} 次抽签 | H0: P(球队进入某组) = 1/16 | df=15")
        print(f"{'=' * 72}")
        print(
            f"  通过卡方检验 ( =0.05):          {n_pass}/{n_total} ({n_pass / n_total:.1%})"
        )
        print(
            f"  通过Bonferroni校正 ( ={alpha_bonf:.4f}): {n_pass_bonf}/{n_total} ({n_pass_bonf / n_total:.1%})"
        )
        print(f"  p值均值:                        {np.mean(all_p):.4f}")
        print(f"  p值中位:                        {np.median(all_p):.4f}")

        failing = [r for r in mres if not r["pass"]]
        if failing:
            print("\n  未通过 =0.05检验的球队:")
            for r in sorted(failing, key=lambda x: x["p_value"]):
                print(
                    f"    {r['name']} ({r['city']}) - ²={r['chi2']:.2f}, p={r['p_value']:.6f}"
                )

        ks_stat, ks_p = kstest(all_p, "uniform")
        print(f"\n  p值分布KS检验: D={ks_stat:.4f}, p={ks_p:.4f}")
        if ks_p >= 0.05:
            print("    -> 与H0一致")
```

```
        else:
            print("    -> 偏离H0")

    return results


if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Compare Q2 draw strategies on the same seed set."
    )
    parser.add_argument(
        "--n-seeds",
        type=int,
        default=100,
        help="number of random seeds for all strategies; look-ahead flow is
            much slower than greedy strategies",
    )
    parser.add_argument(
        "--skip-marginal",
        action="store_true",
        help="skip chi-square marginal uniformity tests",
    )
    args = parser.parse_args()
    run_experiment(n_seeds=args.n_seeds, include_marginal=not args.
        skip_marginal)
```

Listing 3: strategy_comparison.py —— 朴素贪心与前瞻流在 1 000 个配对种子上的策略对比实验，含边际均匀性卡方检验

```
#!/usr/bin/env python3
"""
Generalized Monte Carlo for Q2: basic greedy vs look-ahead flow.

The experiment no longer compares repair or restart variants.  It generates
random province topologies and compares two draw models on the same seeds:

1. basic_greedy: municipal teams are placed randomly; county teams are
    processed
   by descending city size and greedily assigned to the least-conflicting
       group.
2. lookahead_flow: county teams are drawn in random order; each candidate
    group
   is accepted only if the remaining draw can still be completed with
       minimum
   attainable C3 conflict count, certified by max-flow / min-cost flow.
```

```python
"""

from __future__ import annotations

import argparse
import os
import sys
import time
from collections import defaultdict
from dataclasses import dataclass
from typing import Callable, Dict, List, Optional, Tuple

import numpy as np

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from q2_lookahead_flow import Dinic, MinCostMaxFlow


# ============================================================
# 1. Random topology and synthetic team data
# ============================================================


@dataclass
class ProvinceTopology:
    """Synthetic province topology used by generalized Monte Carlo."""

    k: int
    county_per_city: List[int]
    total_teams: int = 64
    num_groups: int = 16
    group_size: int = 4

    @property
    def max_county(self) -> int:
        return max(self.county_per_city) if self.county_per_city else 0

    @property
    def tightness(self) -> float:
        """C3 tightness: max city county-team demand / non-forbidden groups
            ."""
        return self.max_county / (self.num_groups - 1)

    @property
```

```python
    def gini(self) -> float:
        arr = np.array(sorted(self.county_per_city), dtype=float)
        if len(arr) == 0 or arr.sum() == 0:
            return 0.0
        n = len(arr)
        return float((2 * np.arange(1, n + 1) @ arr) / (n * arr.sum()) - (n + 1) / n)


@dataclass(frozen=True)
class Team:
    id: int
    name: str
    city: int
    level: str
    strength: int


@dataclass
class SyntheticTeams:
    teams: List[Team]
    municipal_by_city: Dict[int, int]
    counties_by_city: Dict[int, List[int]]


def generate_random_topology(
    rng: np.random.Generator,
    k: Optional[int] = None,
    concentration: Optional[str] = None,
) -> ProvinceTopology:
    """
    Generate one synthetic province.

    k is restricted to [5, 16], matching a realistic multi-city province and
    keeping C1/C2 feasible.  County-team counts are not capped at 15; therefore
    some concentrated topologies have tau > 1, where C3=0 is mathematically
    impossible and the min-cost-flow objective becomes necessary.
    """

    if k is None:
        k = int(rng.integers(5, 17))
    k = max(5, min(16, k))

```

```
    total_county = 64 - k
    remaining = total_county - k  # one county-level team guaranteed per city

    if concentration is None:
        concentration = str(rng.choice(["uniform", "moderate", "concentrated"]))

    if concentration == "uniform":
        base = total_county // k
        extra = total_county % k
        counts = [base + 1] * extra + [base] * (k - extra)
        rng.shuffle(counts)
        return ProvinceTopology(k=k, county_per_city=counts)

    if concentration == "moderate":
        alpha = np.ones(k) * 2.0
    elif concentration == "concentrated":
        alpha = np.ones(k) * 0.5
        n_big = max(1, k // 3)
        alpha[:n_big] = 3.0
        rng.shuffle(alpha)
    else:
        raise ValueError(f"unknown concentration: {concentration}")

    shares = rng.dirichlet(alpha)
    extra_counts = rng.multinomial(remaining, shares)
    counts = [int(x) + 1 for x in extra_counts]
    return ProvinceTopology(k=k, county_per_city=counts)


def build_teams_from_topology(topo: ProvinceTopology) -> SyntheticTeams:
    teams: List[Team] = []
    municipal_by_city: Dict[int, int] = {}
    counties_by_city: Dict[int, List[int]] = {}

    for city in range(topo.k):
        tid = len(teams)
        municipal_by_city[city] = tid
        teams.append(Team(tid, f"C{city + 1}_M", city, "municipal", 3))

        counties_by_city[city] = []
        n = topo.county_per_city[city]
        n_city_level = n // 2
        for j in range(n):
```

```python
            tid = len(teams)
            level = "county_city" if j < n_city_level else "county"
            strength = 2 if level == "county_city" else 1
            suffix = "CC" if level == "county_city" else "C"
            teams.append(Team(tid, f"C{city + 1}_{suffix}{j + 1}", city, level, strength))
            counties_by_city[city].append(tid)

    return SyntheticTeams(teams, municipal_by_city, counties_by_city)


# ============================================================
# 2. Metrics
# ============================================================


def compute_f1(groups: List[List[int]], instance: SyntheticTeams) -> int:
    total = 0
    for group in groups:
        by_city: Dict[int, int] = defaultdict(int)
        for tid in group:
            team = instance.teams[tid]
            if team.level != "municipal":
                by_city[team.city] += 1
        total += sum(n * (n - 1) // 2 for n in by_city.values())
    return total


def compute_f2(groups: List[List[int]], instance: SyntheticTeams) -> float:
    strengths = [sum(instance.teams[tid].strength for tid in group) for group in groups]
    return float(np.std(strengths))


def compute_f3(groups: List[List[int]], instance: SyntheticTeams) -> float:
    entropies = []
    for group in groups:
        if not group:
            continue
        by_city: Dict[int, int] = defaultdict(int)
        for tid in group:
            by_city[instance.teams[tid].city] += 1
        total = len(group)
        entropies.append(
            -sum((n / total) * np.log(n / total) for n in by_city.values()
```

```
                if n > 0)
        )
    return float(np.mean(entropies)) if entropies else 0.0


# ============================================================
# 3. Basic greedy draw
# ============================================================


def greedy_on_topology(
    topo: ProvinceTopology,
    instance: SyntheticTeams,
    rng: np.random.Generator,
    repair: bool = False,
) -> Tuple[List[List[int]], bool, bool]:
    """
    Basic greedy draw on a synthetic topology.

    The repair argument is accepted only for backward compatibility with
        older
    plotting scripts; it is intentionally ignored in the current experiment
        .
    """

    groups: List[List[int]] = [[] for _ in range(topo.num_groups)]
    mun_group: Dict[int, int] = {}

    cities_for_municipal = list(range(topo.k))
    rng.shuffle(cities_for_municipal)
    slots = np.arange(topo.num_groups)
    rng.shuffle(slots)

    for i, city in enumerate(cities_for_municipal):
        g = int(slots[i])
        groups[g].append(instance.municipal_by_city[city])
        mun_group[city] = g

    city_order = sorted(
        range(topo.k), key=lambda c: topo.county_per_city[c], reverse=True
    )

    for city in city_order:
        forbidden = mun_group[city]
        county_ids = list(instance.counties_by_city[city])
```

```
        rng.shuffle(county_ids)

        for tid in county_ids:
            candidates: List[Tuple[bool, int, int]] = []
            for g in range(topo.num_groups):
                if g == forbidden or len(groups[g]) >= topo.group_size:
                    continue
                has_c3 = any(
                    instance.teams[x].city == city
                    and instance.teams[x].level != "municipal"
                    for x in groups[g]
                )
                candidates.append((has_c3, len(groups[g]), g))

            if not candidates:
                return groups, False, False

            candidates.sort()
            best_c3, best_size, _g = candidates[0]
            top = [g for c3, size, g in candidates if c3 == best_c3 and
                size == best_size]
            groups[int(rng.choice(top))].append(tid)

    return groups, True, False


# ============================================================
# 4. Generic look-ahead flow draw
# ============================================================


@dataclass
class GenericFlowStats:
    flow_checks: int = 0
    min_cost_checks: int = 0
    feasible_candidates: int = 0
    safe_candidates: int = 0
    unsafe_candidates: int = 0
    infeasible_candidates: int = 0
    critical_steps: int = 0
    forced_steps: int = 0
    max_best_final_f1: int = 0


def _remaining_capacity(topo: ProvinceTopology, groups: List[List[int]]) ->
```

```
    List[int]:
    return [topo.group_size - len(group) for group in groups]


def _county_counts_by_city_group(
    topo: ProvinceTopology, instance: SyntheticTeams, groups: List[List[int
        ]]
) -> List[List[int]]:
    counts = [[0] * topo.num_groups for _ in range(topo.k)]
    for g, group in enumerate(groups):
        for tid in group:
            team = instance.teams[tid]
            if team.level != "municipal":
                counts[team.city][g] += 1
    return counts


def _c3_from_counts(counts: List[List[int]]) -> int:
    return sum(n * (n - 1) // 2 for row in counts for n in row)


def _future_zero_c3_feasible(
    topo: ProvinceTopology,
    instance: SyntheticTeams,
    groups: List[List[int]],
    mun_group: Dict[int, int],
    remaining_by_city: List[int],
) -> bool:
    need = sum(remaining_by_city)
    if need == 0:
        return True

    residual = _remaining_capacity(topo, groups)
    if sum(residual) != need:
        return False

    counts = _county_counts_by_city_group(topo, instance, groups)
    if _c3_from_counts(counts) > 0:
        return False

    active = [city for city, n in enumerate(remaining_by_city) if n > 0]
    source = 0
    city_base = 1
    group_base = city_base + len(active)
    sink = group_base + topo.num_groups
```

```
    dinic = Dinic(sink + 1)

    for ci, city in enumerate(active):
        city_node = city_base + ci
        dinic.add_edge(source, city_node, remaining_by_city[city])
        for g in range(topo.num_groups):
            if g == mun_group[city] or residual[g] <= 0:
                continue
            if counts[city][g] == 0:
                dinic.add_edge(city_node, group_base + g, 1)

    for g, cap in enumerate(residual):
        dinic.add_edge(group_base + g, sink, cap)

    return dinic.max_flow(source, sink) == need


def _future_min_c3_cost(
    topo: ProvinceTopology,
    instance: SyntheticTeams,
    groups: List[List[int]],
    mun_group: Dict[int, int],
    remaining_by_city: List[int],
) -> Optional[int]:
    need = sum(remaining_by_city)
    if need == 0:
        return 0

    residual = _remaining_capacity(topo, groups)
    if sum(residual) != need:
        return None

    counts = _county_counts_by_city_group(topo, instance, groups)
    active = [city for city, n in enumerate(remaining_by_city) if n > 0]

    source = 0
    city_base = 1
    group_base = city_base + len(active)
    sink = group_base + topo.num_groups
    mcmf = MinCostMaxFlow(sink + 1)

    for ci, city in enumerate(active):
        city_node = city_base + ci
        demand = remaining_by_city[city]
        mcmf.add_edge(source, city_node, demand, 0)
```

```
        for g in range(topo.num_groups):
            if g == mun_group[city] or residual[g] <= 0:
                continue
            max_units = min(demand, residual[g])
            base_cost = counts[city][g]
            for unit in range(max_units):
                mcmf.add_edge(city_node, group_base + g, 1, base_cost +
                    unit)

    for g, cap in enumerate(residual):
        mcmf.add_edge(group_base + g, sink, cap, 0)

    sent, cost = mcmf.flow(source, sink, need)
    if sent != need:
        return None
    return cost


def _candidate_final_f1(
    topo: ProvinceTopology,
    instance: SyntheticTeams,
    groups: List[List[int]],
    mun_group: Dict[int, int],
    remaining_by_city: List[int],
    tid: int,
    group_id: int,
) -> Optional[Tuple[int, bool]]:
    team = instance.teams[tid]
    if group_id == mun_group[team.city] or len(groups[group_id]) >= topo.
        group_size:
        return None

    groups[group_id].append(tid)

    if _future_zero_c3_feasible(topo, instance, groups, mun_group,
        remaining_by_city):
        groups[group_id].pop()
        return 0, False

    counts = _county_counts_by_city_group(topo, instance, groups)
    current_f1 = _c3_from_counts(counts)
    future_cost = _future_min_c3_cost(
        topo, instance, groups, mun_group, remaining_by_city
    )
    groups[group_id].pop()
```

```python

    if future_cost is None:
        return None
    return current_f1 + future_cost, True


def lookahead_flow_on_topology(
    topo: ProvinceTopology,
    instance: SyntheticTeams,
    rng: np.random.Generator,
) -> Tuple[List[List[int]], bool, GenericFlowStats]:
    groups: List[List[int]] = [[] for _ in range(topo.num_groups)]
    stats = GenericFlowStats()
    mun_group: Dict[int, int] = {}

    cities_for_municipal = list(range(topo.k))
    rng.shuffle(cities_for_municipal)
    slots = np.arange(topo.num_groups)
    rng.shuffle(slots)
    for i, city in enumerate(cities_for_municipal):
        g = int(slots[i])
        groups[g].append(instance.municipal_by_city[city])
        mun_group[city] = g

    county_pool = [tid for ids in instance.counties_by_city.values() for
        tid in ids]
    rng.shuffle(county_pool)
    remaining_by_city = list(topo.county_per_city)

    for tid in county_pool:
        city = instance.teams[tid].city
        remaining_by_city[city] -= 1

        legal_groups = [
            g
            for g in range(topo.num_groups)
            if g != mun_group[city] and len(groups[g]) < topo.group_size
        ]

        scores: List[Tuple[int, int]] = []
        for g in legal_groups:
            result = _candidate_final_f1(
                topo, instance, groups, mun_group, remaining_by_city, tid,
                    g
            )
```

```
            stats.flow_checks += 1
            if result is None:
                stats.infeasible_candidates += 1
                continue
            final_f1, used_min_cost = result
            if used_min_cost:
                stats.min_cost_checks += 1
            scores.append((g, final_f1))

        if not scores:
            return groups, False, stats

        best_final_f1 = min(score for _g, score in scores)
        safe_groups = [g for g, score in scores if score == best_final_f1]
        unsafe_count = len(legal_groups) - len(safe_groups)

        stats.feasible_candidates += len(scores)
        stats.safe_candidates += len(safe_groups)
        stats.unsafe_candidates += unsafe_count
        if unsafe_count > 0:
            stats.critical_steps += 1
        if len(safe_groups) == 1:
            stats.forced_steps += 1
        stats.max_best_final_f1 = max(stats.max_best_final_f1,
            best_final_f1)

        chosen = int(rng.choice(safe_groups))
        groups[chosen].append(tid)

    return groups, True, stats


# ============================================================
# 5. Experiment and reporting
# ============================================================


def _bucket_label(tau: float) -> str:
    if tau <= 0.5:
        return "中等 (  0.5)"
    if tau <= 0.7:
        return "偏紧 (0.5<  0.7)"
    if tau <= 1.0:
        return "紧张 (0.7<  1.0)"
    return "超紧 ( >1.0)"
```

```python


def _mean(values: List[float]) -> float:
    return float(np.mean(values)) if values else 0.0


def _corrcoef_safe(x: np.ndarray, y: np.ndarray) -> float:
    if len(x) < 2 or float(np.std(x)) == 0.0 or float(np.std(y)) == 0.0:
        return float("nan")
    return float(np.corrcoef(x, y)[0, 1])


def run_generalized_mc(
    n_topologies: int = 100,
    n_seeds_per_topo: int = 3,
    seed0: int = 0,
) -> List[dict]:
    print("=" * 88)
    print("  广义蒙特卡洛: 基础贪心 vs 前瞻网络流")
    print(f"  {n_topologies} 个随机拓扑 × {n_seeds_per_topo} 个配对随机种子")
    print("=" * 88)

    results: List[dict] = []

    for ti in range(n_topologies):
        rng_topo = np.random.default_rng(seed0 + ti * 1009)
        topo = generate_random_topology(rng_topo)
        instance = build_teams_from_topology(topo)

        stats = {
            "k": topo.k,
            "max_county": topo.max_county,
            "tightness": topo.tightness,
            "gini": topo.gini,
            "county_per_city": list(topo.county_per_city),
            "greedy_success": 0,
            "greedy_deadlocks": 0,
            "greedy_f1": [],
            "greedy_f2": [],
            "greedy_f3": [],
            "greedy_time": 0.0,
            "flow_success": 0,
            "flow_failures": 0,
            "flow_f1": [],
```

```
            "flow_f2": [],
            "flow_f3": [],
            "flow_checks": [],
            "flow_min_cost_checks": [],
            "flow_unsafe_candidates": [],
            "flow_critical_steps": [],
            "flow_forced_steps": [],
            "flow_time": 0.0,
        }

        for si in range(n_seeds_per_topo):
            seed = seed0 + ti * 100000 + si

            rng_greedy = np.random.default_rng(seed)
            t0 = time.perf_counter()
            g_greedy, ok_greedy, _ = greedy_on_topology(topo, instance,
                rng_greedy)
            stats["greedy_time"] += time.perf_counter() - t0
            if ok_greedy:
                stats["greedy_success"] += 1
                stats["greedy_f1"].append(compute_f1(g_greedy, instance))
                stats["greedy_f2"].append(compute_f2(g_greedy, instance))
                stats["greedy_f3"].append(compute_f3(g_greedy, instance))
            else:
                stats["greedy_deadlocks"] += 1

            rng_flow = np.random.default_rng(seed)
            t0 = time.perf_counter()
            g_flow, ok_flow, flow_stats = lookahead_flow_on_topology(
                topo, instance, rng_flow
            )
            stats["flow_time"] += time.perf_counter() - t0
            if ok_flow:
                stats["flow_success"] += 1
                stats["flow_f1"].append(compute_f1(g_flow, instance))
                stats["flow_f2"].append(compute_f2(g_flow, instance))
                stats["flow_f3"].append(compute_f3(g_flow, instance))
                stats["flow_checks"].append(flow_stats.flow_checks)
                stats["flow_min_cost_checks"].append(flow_stats.
                    min_cost_checks)
                stats["flow_unsafe_candidates"].append(flow_stats.
                    unsafe_candidates)
                stats["flow_critical_steps"].append(flow_stats.
                    critical_steps)
                stats["flow_forced_steps"].append(flow_stats.forced_steps)
```

```
            else:
                stats["flow_failures"] += 1

        results.append(stats)
        if (ti + 1) % max(1, n_topologies // 5) == 0:
            print(f"  已完成 {ti + 1}/{n_topologies} 个拓扑...")

    _print_summary(results, n_seeds_per_topo)
    return results


def _print_summary(results: List[dict], n_seeds_per_topo: int) -> None:
    print(f"\n{'=' * 88}")
    print("  分桶结果")
    print(f"{'=' * 88}")

    bucket_order = [
        "中等 (  0.5)",
        "偏紧 (0.5<  0.7)",
        "紧张 (0.7<  1.0)",
        "超紧 ( >1.0)",
    ]
    buckets: Dict[str, List[dict]] = {label: [] for label in bucket_order}
    for row in results:
        buckets[_bucket_label(row["tightness"])].append(row)

    header = (
        f"  {'紧度区间':<18} {'拓扑数':>6} {'贪心死锁':>10} {'贪心P(F1=0)':>14} "
        f"{'流失败':>8} {'流P(F1=0)':>12} {'流E[F1]':>10} {'流检查/次':>10} "
    )
    print(header)
    print("  " + "-" * 84)

    total_attempts_per_topo = n_seeds_per_topo
    for label in bucket_order:
        bucket = buckets[label]
        if not bucket:
            continue
        n_topo = len(bucket)
        attempts = n_topo * total_attempts_per_topo

        greedy_success = sum(r["greedy_success"] for r in bucket)
        greedy_deadlocks = sum(r["greedy_deadlocks"] for r in bucket)
```

```
        greedy_f1 = [f for r in bucket for f in r["greedy_f1"]]
        greedy_deadlock_rate = greedy_deadlocks / attempts
        greedy_c3_zero = (
            float(np.mean([f == 0 for f in greedy_f1])) if greedy_f1 else 0.0
        )

        flow_success = sum(r["flow_success"] for r in bucket)
        flow_failures = sum(r["flow_failures"] for r in bucket)
        flow_f1 = [f for r in bucket for f in r["flow_f1"]]
        flow_checks = [x for r in bucket for x in r["flow_checks"]]
        flow_fail_rate = flow_failures / attempts
        flow_c3_zero = float(np.mean([f == 0 for f in flow_f1])) if flow_f1 else 0.0
        flow_mean_f1 = _mean([float(f) for f in flow_f1])
        flow_mean_checks = _mean([float(x) for x in flow_checks])

        print(
            f"  {label:<18} {n_topo:>6} {greedy_deadlock_rate:>10.1%} "
            f"{greedy_c3_zero:>14.1%} {flow_fail_rate:>8.1%} "
            f"{flow_c3_zero:>12.1%} {flow_mean_f1:>10.2f} {flow_mean_checks:>10.1f}"
        )

    print(f"\n{'=' * 88}")
    print("  全样本汇总")
    print(f"{'=' * 88}")

    attempts = len(results) * n_seeds_per_topo
    greedy_deadlocks = sum(r["greedy_deadlocks"] for r in results)
    greedy_f1 = [f for r in results for f in r["greedy_f1"]]
    flow_failures = sum(r["flow_failures"] for r in results)
    flow_f1 = [f for r in results for f in r["flow_f1"]]
    flow_checks = [x for r in results for x in r["flow_checks"]]
    flow_min_cost = [x for r in results for x in r["flow_min_cost_checks"]]
    flow_unsafe = [x for r in results for x in r["flow_unsafe_candidates"]]
    flow_time = sum(r["flow_time"] for r in results) / attempts
    greedy_time = sum(r["greedy_time"] for r in results) / attempts

    print(f"  基础贪心死锁率: {greedy_deadlocks / attempts:.2%}")
    print(
        f"  基础贪心成功样本 P(F1=0): "
        f"{np.mean([f == 0 for f in greedy_f1]) if greedy_f1 else 0:.2%}"
    )
    print(f"  前瞻网络流失败率: {flow_failures / attempts:.2%}")
```

```
    print(
        f"  前瞻网络流 P(F1=0): "
        f"{np.mean([f == 0 for f in flow_f1]) if flow_f1 else 0:.2%}"
    )
    print(f"  前瞻网络流 E[F1]: {_mean([float(f) for f in flow_f1]):.4f}")
    print(f"  前瞻网络流平均检查候选组: {_mean([float(x) for x in flow_checks]):.1f}")
    print(f"  前瞻网络流平均最小费用流回退: {_mean([float(x) for x in flow_min_cost]):.1f}")
    print(f"  前瞻网络流平均危险候选组: {_mean([float(x) for x in flow_unsafe]):.1f}")
    print(f"  基础贪心平均耗时: {greedy_time:.4f}s/次")
    print(f"  前瞻网络流平均耗时: {flow_time:.4f}s/次")

    tightness = np.array([r["tightness"] for r in results])
    greedy_deadlock_by_topo = np.array(
        [r["greedy_deadlocks"] / n_seeds_per_topo for r in results]
    )
    flow_mean_f1_by_topo = np.array(
        [_mean([float(f) for f in r["flow_f1"]]) for r in results]
    )
    if len(results) >= 2:
        corr_deadlock = _corrcoef_safe(tightness, greedy_deadlock_by_topo)
        corr_flow_f1 = _corrcoef_safe(tightness, flow_mean_f1_by_topo)
        print(f"    与贪心死锁率相关系数: {corr_deadlock:.4f}")
        print(f"    与前瞻流E[F1]相关系数: {corr_flow_f1:.4f}")

    zj = ProvinceTopology(k=11, county_per_city=[3, 4, 8, 5, 3, 3, 7, 4, 2, 6, 8])
    tighter = sum(1 for r in results if r["tightness"] > zj.tightness)
    print(
        f"  浙江省参考: k={zj.k}, max(n_i)={zj.max_county}, "
        f" ={zj.tightness:.3f}; 比浙江更紧的拓扑 {tighter}/{len(results)}"
    )


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--n-topologies", type=int, default=100)
    parser.add_argument("--n-seeds", type=int, default=3)
    parser.add_argument("--seed0", type=int, default=0)
    args = parser.parse_args()
    run_generalized_mc(
        n_topologies=args.n_topologies,
        n_seeds_per_topo=args.n_seeds,
```

```
        seed0=args.seed0,
    )


if __name__ == "__main__":
    main()
```

Listing 4: generalized_mc.py —— 随机行政拓扑生成与基础贪心、前瞻网络流的广义蒙特卡洛鲁棒性验证

B.3 问题 3：比赛地点选择

```
#!/usr/bin/env python3
"""
Q3: 浙超小组赛比赛地点选择
==========================
从 64 个参赛单位中选 8 个作为小组赛场地，每个场地承办 2 个小组.
目标: 最小化各队总旅行距离/时间，同时保证地理覆盖的公平性.

模型: 设施选址 (p-median) + 指派问题联合优化 (ILP).

支持的成本模型:
  A. haversine       — 大圆距离 (km)，基准
  B. road_time       — 公路旅行时间 (min)
  C. railway_time    — 铁路旅行时间 (min)
  D. combined_time   — 公路+铁路平均 (min)
  E. fan_weighted    — 公路时间 × 人口权重 (万人·min)
  F. combined_top20  — 综合时间，仅从影响力Top20候选场地中选择
"""

import sys
import os
import json
import csv
import numpy as np
from math import radians, sin, cos, asin, sqrt
from collections import defaultdict

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from q1_grouping import (
    TEAMS,
    TEAM_INDEX,
    NUM_GROUPS,
```

```
    GROUP_SIZE,
    scheme_d_ilp_balanced,
    scheme_b_serpentine,
    check_c1,
    check_c2,
    check_c3,
)

# ============================================================
# 1. 地理坐标与距离
# ============================================================


def load_coords_from_jsonl(filepath=None):
    """从 JSONL 文件加载坐标，返回 {name: (lat, lon)}."""
    if filepath is None:
        filepath = os.path.join(
            os.path.dirname(os.path.abspath(__file__)),
            "conditions_q3", "坐标", "zhejiang_coords_per_line.json",
        )
    coords = {}
    with open(filepath, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            obj = json.loads(line)
            lon, lat = obj["gdm"]
            coords[obj["name"]] = (lat, lon)
    aliases = {"景宁畲族自治县": "景宁县"}
    for source, target in aliases.items():
        if source in coords and target not in coords:
            coords[target] = coords[source]
    return coords


COORDS = load_coords_from_jsonl()


def haversine(name1, name2):
    """计算两个地点之间的大圆距离 (km)."""
    lat1, lon1 = map(radians, COORDS[name1])
    lat2, lon2 = map(radians, COORDS[name2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
```

```python
    a = sin(dlat / 2) ** 2 + cos(lat1) * cos(lat2) * sin(dlon / 2) ** 2
    return 6371 * 2 * asin(sqrt(min(a, 1.0)))


def build_distance_matrix(teams):
    """构建所有球队之间的 Haversine 距离矩阵."""
    n = len(teams)
    D = np.zeros((n, n))
    for i in range(n):
        for j in range(i + 1, n):
            d = haversine(teams[i].name, teams[j].name)
            D[i, j] = D[j, i] = d
    return D


# ============================================================
# 2. 外部数据加载 (交通、人口、收入)
# ============================================================

DATA_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "conditions_q3")


def _resolve_name(name):
    """统一名称映射: '景宁县'  '景宁畲族自治县'"""
    if name == "景宁县":
        return "景宁畲族自治县"
    if name == "景宁畲族自治县":
        return "景宁县"
    return name


def _load_travel_time_matrix(filename):
    """加载旅行时间矩阵 (CSV). 返回 (names, matrix)."""
    filepath = os.path.join(DATA_DIR, "交通", filename)
    with open(filepath, "r", encoding="utf-8") as f:
        reader = csv.reader(f)
        header = next(reader)
        names = header[1:]
        n = len(names)
        matrix = np.zeros((n, n))
        for i, row in enumerate(reader):
            matrix[i, :] = [float(x) for x in row[1:]]
    return names, matrix

```

```python

def _load_jsonl(filepath):
    """加载 JSONL 文件，返回 {name: value_dict}."""
    data = {}
    with open(filepath, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            obj = json.loads(line)
            data[obj["name"]] = obj
    return data


def load_all_data():
    """加载所有外部数据，建立统一的查找字典."""
    road_names, road_matrix = _load_travel_time_matrix("travel_time_matrix_minutes.csv")
    rail_names, rail_matrix = _load_travel_time_matrix("railway_travel_time_matrix_minutes.csv")

    road_idx = {}
    for i, name in enumerate(road_names):
        road_idx[name] = i
        road_idx[_resolve_name(name)] = i

    rail_idx = {}
    for i, name in enumerate(rail_names):
        rail_idx[name] = i
        rail_idx[_resolve_name(name)] = i

    income = {}
    income_data = _load_jsonl(os.path.join(DATA_DIR, "可支配收入", "可支配收入.json"))
    for name, obj in income_data.items():
        income[name] = obj["可支配收入"]
        income[_resolve_name(name)] = obj["可支配收入"]

    population = {}
    pop_data = _load_jsonl(os.path.join(DATA_DIR, "常住人口", "常住人口（万人）.json"))
    for name, obj in pop_data.items():
        population[name] = obj["常住人口"]
        population[_resolve_name(name)] = obj["常住人口"]

```

```python
    return {
        "road_names": road_names, "road_matrix": road_matrix, "road_idx": road_idx,
        "rail_names": rail_names, "rail_matrix": rail_matrix, "rail_idx": rail_idx,
        "income": income, "population": population,
    }


def build_influence_scores(teams, data):
    """计算影响力得分 I_k = 0.5 * pop/max + 0.5 * income/max."""
    pop = data["population"]
    inc = data["income"]
    pop_max = max(pop[t.name] for t in teams if t.name in pop)
    inc_max = max(inc[t.name] for t in teams if t.name in inc)
    scores = {}
    for idx, team in enumerate(teams):
        p = pop.get(team.name, 0.0) / pop_max
        q = inc.get(team.name, 0.0) / inc_max
        scores[idx] = 0.5 * p + 0.5 * q
    return scores


def select_influence_candidates(teams, data, top_n=20):
    """按影响力得分筛选前 top_n 个候选场地."""
    scores = build_influence_scores(teams, data)
    return [idx for idx, _ in sorted(scores.items(), key=lambda x: -x[1])[:top_n]]


# ============================================================
# 3. 成本矩阵构建
# ============================================================


def build_per_team_cost_matrix(teams, model="haversine", data=None):
    """
    构建 C[i, k] = 球队 i 到场地 k 的单队成本矩阵.
    支持模型: haversine, road_time, railway_time, combined_time, fan_weighted.
    """
    n = len(teams)
    C = np.zeros((n, n))

    if model == "haversine":
```

```
        for i in range(n):
            for k in range(n):
                C[i, k] = haversine(teams[i].name, teams[k].name)

    elif model in ("road_time", "railway_time", "combined_time"):
        if data is None:
            raise ValueError("交通模型需要传入 data 参数")
        rmat = data["road_matrix"]
        lmat = data["rail_matrix"]
        ridx = data["road_idx"]
        lidx = data["rail_idx"]
        for i in range(n):
            for k in range(n):
                kn = teams[k].name
                inn = teams[i].name
                if model == "road_time":
                    C[i, k] = rmat[ridx[inn], ridx[kn]]
                elif model == "railway_time":
                    C[i, k] = lmat[lidx[inn], lidx[kn]]
                else:  # combined_time
                    C[i, k] = 0.5 * (
                        rmat[ridx[inn], ridx[kn]] + lmat[lidx[inn], lidx[kn]]
                    )

    elif model == "fan_weighted":
        if data is None:
            raise ValueError("人口加权模型需要传入 data 参数")
        rmat = data["road_matrix"]
        ridx = data["road_idx"]
        pop = data["population"]
        for i in range(n):
            for k in range(n):
                kn = teams[k].name
                inn = teams[i].name
                C[i, k] = rmat[ridx[inn], ridx[kn]] * pop.get(inn, 50.0)

    else:
        raise ValueError(f"未知成本模型: {model}")

    return C


def build_group_cost_matrix(groups, teams, C):
    """从单队成本矩阵汇总得到 D[g, k] = sum_i_in_g C[i, k]."""
```

```python
    team_idx = {t.name: i for i, t in enumerate(teams)}
    n_groups = len(groups)
    n_locs = len(teams)
    D = np.zeros((n_groups, n_locs))
    for g in range(n_groups):
        for k in range(n_locs):
            D[g, k] = sum(C[team_idx[name], k] for name in groups[g])
    return D, team_idx


# ============================================================
# 4. ILP 选址模型
# ============================================================


def _group_has_home_venue(group, venue_name):
    return venue_name in group


def ilp_venue_selection(groups, teams, D, time_limit=60,
                        candidate_indices=None, forbid_home=True):
    """
    ILP 联合优化选址 + 指派: 选 8 个场地并将 16 个小组分配到场地,
    每个场地恰好承办 2 个小组, 最小化总成本 D[g, k].

    返回: (venues, assignment, total_cost) 或 (None, None, None).
    """
    from pulp import (
        LpMinimize, LpProblem, LpVariable, lpSum,
        LpBinary, LpStatus, value, PULP_CBC_CMD,
    )

    n_loc = len(teams)
    n_groups = len(groups)
    loc_names = [t.name for t in teams]
    candidate_set = set(candidate_indices) if candidate_indices is not None
        else set(range(n_loc))

    prob = LpProblem("VenueSelection", LpMinimize)

    y = {k: LpVariable(f"y_{k}", cat=LpBinary) for k in range(n_loc)}
    z = {(g, k): LpVariable(f"z_{g}_{k}", cat=LpBinary)
         for g in range(n_groups) for k in range(n_loc)}

    # 恰好选 8 个场地
```

```python
    prob += lpSum(y[k] for k in range(n_loc)) == 8

    # 候选集约束
    for k in range(n_loc):
        if k not in candidate_set:
            prob += y[k] == 0

    # 每个小组恰好指派到 1 个场地
    for g in range(n_groups):
        prob += lpSum(z[g, k] for k in range(n_loc)) == 1

    # 只能指派到已选场地
    for g in range(n_groups):
        for k in range(n_loc):
            prob += z[g, k] <= y[k]

    # 主场禁止
    if forbid_home:
        for g in range(n_groups):
            for k in range(n_loc):
                if _group_has_home_venue(groups[g], loc_names[k]):
                    prob += z[g, k] == 0

    # 每个场地恰好 2 个小组
    for k in range(n_loc):
        prob += lpSum(z[g, k] for g in range(n_groups)) == 2 * y[k]

    # 目标: 最小化总成本
    prob += lpSum(D[g, k] * z[g, k] for g in range(n_groups) for k in range
        (n_loc))

    prob.solve(PULP_CBC_CMD(msg=0, timeLimit=time_limit))

    if LpStatus[prob.status] != "Optimal":
        return None, None, None

    venues = [k for k in range(n_loc) if value(y[k]) > 0.5]
    assignment = {}
    for g in range(n_groups):
        for k in range(n_loc):
            if value(z[g, k]) > 0.5:
                assignment[g] = k
                break

    total_cost = sum(D[g, assignment[g]] for g in range(n_groups))
```

```python
    return venues, assignment, total_cost


def assignment_ilp(groups, teams, venue_indices, C=None, forbid_home=True):
    """
    给定场地集合，用 ILP 将 16 个小组最优分配到场地（每场地恰好 2 组）.
    C 为单队成本矩阵（若不传则用 Haversine 距离）.
    返回：(assignment, total_cost).
    """
    from pulp import (
        LpMinimize, LpProblem, LpVariable, lpSum,
        LpBinary, value, PULP_CBC_CMD,
    )

    n_groups = len(groups)
    loc_names = [t.name for t in teams]
    n_venues = len(venue_indices)

    if C is None:
        C = build_per_team_cost_matrix(teams, model="haversine")
    D, _ = build_group_cost_matrix(groups, teams, C)

    prob = LpProblem("AssignOnly", LpMinimize)
    z = {(g, vidx): LpVariable(f"za_{g}_{vidx}", cat=LpBinary)
         for g in range(n_groups) for vidx in range(n_venues)}

    for g in range(n_groups):
        prob += lpSum(z[g, vidx] for vidx in range(n_venues)) == 1
    for vidx in range(n_venues):
        prob += lpSum(z[g, vidx] for g in range(n_groups)) == 2
    if forbid_home:
        for g in range(n_groups):
            for vidx, k in enumerate(venue_indices):
                if _group_has_home_venue(groups[g], loc_names[k]):
                    prob += z[g, vidx] == 0

    prob += lpSum(D[g, k] * z[g, vidx]
                  for g in range(n_groups)
                  for vidx, k in enumerate(venue_indices))

    prob.solve(PULP_CBC_CMD(msg=0))

    assignment = {}
    for g in range(n_groups):
        for vidx, k in enumerate(venue_indices):
```

```
            if value(z[g, vidx]) > 0.5:
                assignment[g] = k
                break

    total = sum(D[g, assignment[g]] for g in range(n_groups))
    return assignment, total


def heuristic_venues(teams):
    """
    启发式选址：地理分散 + 贪心。
    先选总距离最小的点，然后每次选离已选点集最远且成本合理的点。
    """
    loc_names = [t.name for t in teams]
    cost = {loc: sum(haversine(loc, t.name) for t in teams) for loc in
        loc_names}
    sorted_locs = sorted(cost.keys(), key=lambda x: cost[x])
    selected = [sorted_locs[0]]

    for _ in range(7):
        best_score = -1
        best_loc = None
        for loc in sorted_locs:
            if loc in selected:
                continue
            min_dist = min(haversine(loc, s) for s in selected)
            score = min_dist / (cost[loc] + 1)
            if score > best_score:
                best_score = score
                best_loc = loc
        selected.append(best_loc)

    return selected


# ============================================================
# 5. 结果输出
# ============================================================


def print_venue_result(groups, teams, venues, assignment, total_cost,
                       C, label, unit):
    """打印单次选址结果。"""
    loc_names = [t.name for t in teams]
    team_idx_map = {t.name: i for i, t in enumerate(teams)}
```

```python

    print(f"\n{' ' * 72}")
    print(f"  {label}")
    print(f"{' ' * 72}")

    venue_groups = defaultdict(list)
    for g, k in assignment.items():
        venue_groups[k].append(g)

    for vi, k in enumerate(sorted(venues)):
        vname = loc_names[k]
        gs = venue_groups.get(k, [])
        lat, lon = COORDS.get(vname, (0, 0))
        print(f"\n  场地{vi + 1}: {vname} ({lat:.2f}°N, {lon:.2f}°E)")
        for g in gs:
            parts = []
            for name in groups[g]:
                ti = team_idx_map.get(name)
                cost = C[ti, k] if ti is not None else 0
                parts.append(f"{name}({cost:.0f}{unit})")
            print(f"    组{g + 1:2d}: {', '.join(parts)}")

    # 统计
    actual_costs = []
    for g, k in assignment.items():
        for name in groups[g]:
            ti = team_idx_map.get(name)
            if ti is not None:
                actual_costs.append(C[ti, k])
    avg_actual = np.mean(actual_costs) if actual_costs else 0
    max_actual = max(actual_costs) if actual_costs else 0
    min_actual = min(actual_costs) if actual_costs else 0

    # Haversine 参考距离
    dists = []
    for g, k in assignment.items():
        for name in groups[g]:
            dists.append(haversine(name, loc_names[k]))

    print(f"\n  总成本: {total_cost:.0f} {unit}")
    print(f"  平均每队: {avg_actual:.1f}{unit}  |  "
          f"最远: {max_actual:.0f}{unit}  |  最近: {min_actual:.0f}{unit}")
    print(f"  Haversine 参考: {sum(dists):.0f} km, "
          f"平均 {np.mean(dists):.1f} km, 最远 {max(dists):.0f} km")

```

```python
    # 地理覆盖
    venue_coords = [COORDS.get(loc_names[k], (0, 0)) for k in venues]
    lats = [c[0] for c in venue_coords]
    lons = [c[1] for c in venue_coords]
    print(f"  纬度范围: {min(lats):.2f}° ~ {max(lats):.2f}°  "
          f"经度范围: {min(lons):.2f}° ~ {max(lons):.2f}°")


def print_comparison_table(results, ref_label="haversine"):
    """打印所有成本模型的横向对比表."""
    print(f"\n{'=' * 72}")
    print(f"  成本模型横向对比")
    print(f"{'=' * 72}")

    header = (f"  {'模型':<24} {'总成本':<14} {'单位':<8} "
              f"{'平均/队':<14} {'最远/队':<12} {'Haversine参考':<14} {'场地交集'}")
    print(header)
    print(f"  {' ' * 86}")

    ref_venues = None
    for r in results:
        if r["label"] == ref_label:
            ref_venues = set(r.get("venues", []))
            break

    for r in results:
        name = r["label"]
        unit = r["unit"]
        tc = r["total_cost"]
        avg = tc / 64
        mc = r["max_per_team"]
        hr = r.get("haversine_ref", 0)

        jaccard = ""
        if ref_venues is not None and r.get("venues"):
            common = len(ref_venues & set(r["venues"]))
            jaccard = f"{common}/8"

        print(f"  {name:<24} {tc:<14.0f} {unit:<8} "
              f"{avg:<14.1f} {mc:<12.0f} {hr:<14.0f} {jaccard}")


# ============================================================
# MAIN
```

```python
# ============================================================


def main():
    print("=" * 72)
    print("  浙超小组赛选址 — 问题3: 比赛地点选择")
    print("=" * 72)

    # ---- 分组 ----
    groups = scheme_d_ilp_balanced(w_c3=10, w_str=1)
    if groups is None:
        groups = scheme_b_serpentine(seed=42)
        print("\n  使用蛇形分组 (ILP均衡不可用)")
    else:
        print("\n  使用 ILP均衡分组")

    c1v, c2v = check_c1(groups), check_c2(groups)
    c3n, _ = check_c3(groups)
    print(f"  C1={'通过' if not c1v else '违反'} "
          f"C2={'通过' if not c2v else '违反'} "
          f"C3={'通过' if c3n == 0 else f'{c3n}对冲突'}")

    # ---- 加载外部数据 ----
    print("\n  加载外部数据...")
    data = load_all_data()
    print(f"    公路旅行矩阵: {data['road_matrix'].shape}")
    print(f"    铁路旅行矩阵: {data['rail_matrix'].shape}")
    print(f"    收入数据:     {len(data['income'])} 条")
    print(f"    人口数据:     {len(data['population'])} 条")

    influence_top20 = select_influence_candidates(TEAMS, data, top_n=20)
    loc_names = [t.name for t in TEAMS]
    print("\n  影响力Top20候选场地 (人口+收入各50%):")
    print("    " + "、".join(loc_names[i] for i in influence_top20))

    # ---- 实验配置 ----
    models = [
        ("haversine", "km"), ("road_time", "min"), ("railway_time", "min"),
        ("combined_time", "min"), ("fan_weighted", "万人·min"),
    ]
    experiments = [
        {"model": m, "unit": u, "label": m, "candidate_indices": None}
        for m, u in models
    ]
    experiments.append({
```

```
        "model": "combined_time", "unit": "min", "label": "combined_top20",
        "candidate_indices": influence_top20,
    })

    # ---- 逐模型求解 ----
    results = []
    for exp in experiments:
        model_name = exp["model"]
        unit = exp["unit"]
        label = exp["label"]
        candidate_indices = exp["candidate_indices"]

        print(f"\n  构建成本矩阵: {label} ...")
        C = build_per_team_cost_matrix(TEAMS, model=model_name, data=data)
        D, team_idx_map = build_group_cost_matrix(groups, TEAMS, C)
        print(f"    范围: [{D.min():.0f}, {D.max():.0f}] {unit}")

        print(f"    求解 ILP ...")
        time_limit = 120 if model_name == "fan_weighted" else 60
        venues, assignment, total_cost = ilp_venue_selection(
            groups, TEAMS, D, time_limit=time_limit,
            candidate_indices=candidate_indices, forbid_home=True,
        )

        if venues is None:
            print(f"    [警告] {label} ILP 未找到最优解")
            continue

        hav_dist = sum(
            haversine(name, loc_names[assignment[g]])
            for g in range(NUM_GROUPS) for name in groups[g]
        )
        max_per_team = max(
            C[team_idx_map[name], assignment[g]]
            for g in range(NUM_GROUPS) for name in groups[g]
        )

        results.append({
            "model": model_name, "label": label, "unit": unit,
            "total_cost": total_cost, "max_per_team": max_per_team,
            "haversine_ref": hav_dist, "venues": venues, "assignment":
                assignment,
        })

        print_venue_result(groups, TEAMS, venues, assignment, total_cost,
```

```
                                C, label, unit)

    # ---- 横向对比 ----
    print_comparison_table(results)

    # ---- 选址一致性 ----
    if len(results) >= 2:
        print(f"\n{'=' * 72}")
        print(f"  选址一致性分析")
        print(f"{'=' * 72}")
        all_venue_sets = [set(r.get("venues", [])) for r in results]

        print(f"\n  各模型选出的 8 个场地间的交集:")
        for i in range(len(results)):
            for j in range(i + 1, len(results)):
                common = len(all_venue_sets[i] & all_venue_sets[j])
                print(f"    {results[i]['label']:>16}  {results[j]['label
                    ']:<16} = {common}/8")

        venue_freq = defaultdict(int)
        for vs in all_venue_sets:
            for v in vs:
                venue_freq[v] += 1
        popular = sorted(venue_freq.items(), key=lambda x: -x[1])
        print(f"\n  高频场地 (被多个模型选中):")
        for v, f in popular:
            if f >= 2:
                print(f"    {loc_names[v]:10s}  被 {f}/{len(results)} 个模
                    型选中")

    # ---- 结论 ----
    if results:
        print(f"\n{'=' * 72}")
        print(f"  结论")
        print(f"{'=' * 72}")
        base = results[0]
        for r in results[1:]:
            print(f"\n  {r['label']} vs haversine (基准):")
            print(f"    总成本: {r['total_cost']:.0f} {r['unit']} "
                  f"(Haversine距离参考: {r['haversine_ref']:.0f} km "
                  f"vs 基准 {base['haversine_ref']:.0f} km)")
            if r["haversine_ref"] > 0:
                pct = ((r["haversine_ref"] - base["haversine_ref"])
                       / base["haversine_ref"] * 100)
```

```python
                print(f"    用此模型选出的场地，在Haversine距离下多走了 {pct:+.1f}% 的距离")

    # ---- 启发式选址对比 ----
    print(f"\n{'=' * 72}")
    print(f"  ILP 最优 vs 启发式贪心 对比")
    print(f"{'=' * 72}")

    h_locs = heuristic_venues(TEAMS)
    h_venue_indices = [loc_names.index(l) for l in h_locs]
    C_hav = build_per_team_cost_matrix(TEAMS, model="haversine")
    h_assign, h_cost = assignment_ilp(groups, TEAMS, h_venue_indices, C=C_hav)
    print_venue_result(groups, TEAMS, h_venue_indices, h_assign, h_cost,
                       C_hav, "启发式选址 (地理分散贪心 + 最优指派)", "km")

    if results:
        ilp_km = results[0]["total_cost"]
        gap = (h_cost - ilp_km) / ilp_km * 100
        print(f"\n  启发式比 ILP 最优多 {gap:.1f}% 总距离，说明联合优化效果显著.")


if __name__ == "__main__":
    main()
```

Listing 5: q3_venue.py —— Haversine 距离、公路/铁路旅行时间、人口加权等多维成本模型的 ILP 设施选址，含主场禁止约束和启发式对比

B.4 问题 4：赛制仿真

```python
#!/usr/bin/env python3
"""
Q4: 浙超赛制合理化建议 — 蒙特卡洛仿真
========================================
基于 Bradley-Terry 模型模拟小组赛 + 淘汰赛，
定量评价当前赛制的公平性，并对比多种赛制变体.

评价指标:
  - 公平性: 实力排名与最终名次的 Spearman 相关系数
  - 晋级准确率: 实力前 32 名实际晋级的比例
  - 冠军集中度: 实力 Top-1 的夺冠概率
  - 稳健性: 层内正态扰动 s_i ~ N(s_level, (1/3)^2)
"""
```

```python

import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import numpy as np
from collections import defaultdict
from scipy import stats
from q1_grouping import (
    TEAMS,
    TEAM_INDEX,
    check_c1,
    check_c2,
    check_c3,
)


# 方案D: ILP(均衡), 来自 q1_output.txt.
# Q4只评价赛制, 固定分组可避免每次运行都重新求解ILP。
BALANCED_GROUPS = [
    ["龙港市", "德清县", "东阳市", "三门县"],
    ["嘉善县", "金华市", "岱山县", "庆元县"],
    ["永嘉县", "平湖市", "长兴县", "舟山市"],
    ["温州市", "海盐县", "诸暨市", "磐安县"],
    ["余姚市", "安吉县", "浦江县", "丽水市"],
    ["杭州市", "象山县", "兰溪市", "常山县"],
    ["宁波市", "江山市", "仙居县", "青田县"],
    ["建德市", "苍南县", "龙游县", "温岭市"],
    ["武义县", "衢州市", "玉环市", "云和县"],
    ["文成县", "绍兴市", "永康市", "开化县"],
    ["宁海县", "嘉兴市", "义乌市", "遂昌县"],
    ["瑞安市", "海宁市", "新昌县", "松阳县"],
    ["桐庐县", "乐清市", "临海市", "景宁县"],
    ["淳安县", "慈溪市", "台州市", "缙云县"],
    ["平阳县", "桐乡市", "嵊州市", "龙泉市"],
    ["泰顺县", "湖州市", "嵊泗县", "天台县"],
]

LAYER_SIGMA = 1 / 3
MIN_STRENGTH = 0.05


# ============================================================
# 1. Bradley-Terry 比赛模拟
```

```python
# ============================================================

# [问题] 需要一个合理的比赛结果模型. 足球比赛有平局, 但建模竞赛中
#        可以简化为"无平局"以聚焦赛制分析.
# [解决] 采用 Bradley-Terry 模型 P(i胜j) = s_i/(s_i+s_j),
#        不考虑平局. 小组赛积分: 胜=3分, 负=0分.
#        如果需要更真实的模型, 可引入随机进球数 (Poisson).

def draw_layered_strengths(rng, sigma=LAYER_SIGMA, min_strength=MIN_STRENGTH):
    """
    层内正态实力模型.
    以 3/2/1 为层级均值, 每次赛事先抽取一组潜在真实实力.
    为保证 Bradley-Terry 概率有效, 对极小概率的非正抽样做截断重抽.
    """
    means = np.array([t.strength for t in TEAMS], dtype=float)
    strengths = rng.normal(means, sigma)
    invalid = strengths <= min_strength
    while np.any(invalid):
        strengths[invalid] = rng.normal(means[invalid], sigma)
        invalid = strengths <= min_strength
    return strengths


def match_result(rng, strength_i, strength_j):
    """
    Bradley-Terry 单场结果.
    返回: (i得分, j得分) — 小组赛中 胜=3, 负=0.
    """
    p_i_win = strength_i / (strength_i + strength_j)
    if rng.random() < p_i_win:
        return 3, 0
    else:
        return 0, 3


def generate_goals(rng, s_i, s_j):
    """
    Poisson 进球数, 同时依赖攻方和守方实力.
    lambda_ij = 0.5 + 0.2*s_i - 0.1*s_j, 保证 >= 0.1.
    仅用于同分排序, 不影响 BT 胜负概率.
    """
    lam = max(0.1, 0.5 + 0.2 * s_i - 0.1 * s_j)
    return rng.poisson(lam)

```

```python

def simulate_group_stage(rng, groups, team_strengths):
    """
    模拟小组赛（单循环）.
    每组 4 队，共 6 场：(0,1)(0,2)(0,3)(1,2)(1,3)(2,3).
    返回：每组按积分降序排列的球队索引列表.
    """
    group_rankings = []

    for group in groups:
        indices = [TEAM_INDEX[name] for name in group]
        points = defaultdict(int)
        goals_for = defaultdict(int)  # 进球数（用于排名）

        for a, b in [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]:
            ia, ib = indices[a], indices[b]
            sa, sb = match_result(rng, team_strengths[ia], team_strengths[ib])
            points[ia] += sa
            points[ib] += sb
            # 模拟进球数（Poisson，依赖攻守双方实力）
            gf_a = generate_goals(rng, team_strengths[ia], team_strengths[ib])
            gf_b = generate_goals(rng, team_strengths[ib], team_strengths[ia])
            goals_for[ia] += gf_a
            goals_for[ib] += gf_b

        # [问题] 小组积分相同时如何排名？
        # [解决] 依次比较：积分 → 净胜球（用进球数近似）→ 随机抽签.
        ranked = sorted(indices, key=lambda i: (points[i], goals_for[i], rng.random()),
                        reverse=True)
        group_rankings.append(ranked)

    return group_rankings


def simulate_knockout(rng, team_indices, team_strengths):
    """
    模拟淘汰赛（单场淘汰）.
    返回：(冠军index，最终排名列表 — 从第1名到第32名).
    """
    current_round = list(team_indices)  # 32 支队

```

```python
    # [问题] 淘汰赛的对阵如何确定?
    # [解决] 简化为: 每轮随机配对 (实际赛事会根据小组排名设置种子,
    #         但为了分析赛制本身的公平性, 随机配对更中性).
    #         更细的同组回避和种子蛇形对阵作为改进建议讨论.
    rng.shuffle(current_round)

    round_names = ["32强", "16强", "八强", "四强", "决赛"]
    eliminated_by_round = []

    for rnd, name in enumerate(round_names):
        next_round = []
        round_losers = []
        for i in range(0, len(current_round), 2):
            ia, ib = current_round[i], current_round[i + 1]
            sa, sb = match_result(rng, team_strengths[ia], team_strengths[
                ib])
            if sa > sb:
                winner, loser = ia, ib
            else:
                winner, loser = ib, ia
            next_round.append(winner)
            round_losers.append(loser)
        current_round = next_round
        eliminated_by_round.append(round_losers)
        if len(current_round) > 1:
            rng.shuffle(current_round)

    # 冠军
    champion = current_round[0]
    # 排名: 冠军 → 决赛败者 → 半决赛败者 → ... → 32强败者.
    # 同一轮被淘汰的球队只产生并列区间, 用当轮随机对阵顺序补位,
    # 不再使用真实实力排序, 以免人为抬高 Spearman 指标.
    final_ranking = [champion]
    for losers in reversed(eliminated_by_round):
        final_ranking.extend(losers)
    return champion, final_ranking


def complete_group_loser_ranking(rng, group_rankings):
    """
    小组未出线球队的第33--64名补位.

    真实赛制不会给所有小组第3/第4名产生精确总排名。本文按小组名次分层:
    16个小组第3名排在16个小组第4名前; 同层球队随机补位。该规则只使用
    比赛阶段可观察信息, 不使用真实实力排序。
```

```python
    """
    group_losers = []
    for pos in (2, 3):
        layer = [ranking[pos] for ranking in group_rankings]
        rng.shuffle(layer)
        group_losers.extend(layer)
    return group_losers


def advancement_reference(strength_rank, team_strengths, cutoff=32):
    """
    构造“应晋级”基准集合。

    层内随机模型有唯一实力排序，基准就是Top-32。固定3/2/1层级模型存在
    大量并列，不存在唯一第32名；本文改用rank<=32所对应的高/中档球队集合，
    并用该集合真实规模作分母，避免把31队基准误除以32。
    """
    rank = np.asarray(strength_rank)
    mask = rank <= cutoff
    denominator = int(np.sum(mask))
    if denominator == 0:
        raise ValueError("advancement reference set is empty")

    strengths = np.asarray(team_strengths, dtype=float)
    if denominator == cutoff and len(np.unique(strengths)) == len(strengths
        ):
        label = "实力Top-32实际晋级比例"
    elif denominator == 31 and len(np.unique(strengths)) == 3:
        label = "高/中档球队实际晋级比例，31队基准"
    else:
        label = f"rank<=32基准球队实际晋级比例，{denominator}队基准"
    return mask, denominator, label


# ============================================================
# 2. 完整赛制仿真
# ============================================================

def simulate_tournament(rng, groups, team_strengths):
    """运行一次完整赛制（小组赛 + 淘汰赛），返回最终排名"""
    group_rankings = simulate_group_stage(rng, groups, team_strengths)

    # 小组前 2 名晋级
    qualified = []
    for ranking in group_rankings:
```

```
        qualified.extend(ranking[:2])

    champion, final_ranking = simulate_knockout(rng, qualified,
        team_strengths)

    # 完整排名 (64 队): 淘汰赛排名 + 小组赛淘汰的 32 队
    knockout_ranking = final_ranking  # 32 队排名
    group_losers = complete_group_loser_ranking(rng, group_rankings)

    full_ranking = knockout_ranking + group_losers
    return full_ranking


def compute_metrics(rng, groups, team_strengths, n_sim):
    """蒙特卡洛仿真, 计算多项公平性指标"""
    strength_rank = stats.rankdata([-s for s in team_strengths])  # 实力排
        名 (1=最强)
    advancement_mask, advancement_denominator, advancement_label = (
        advancement_reference(strength_rank, team_strengths)
    )

    spearmans = []
    top32_rates = []
    champion_top1 = 0
    champion_top4 = 0
    champion_top8 = 0
    upsets_group = 0
    upsets_knockout = 0
    total_group_matches = 0
    total_knockout_matches = 0

    for _ in range(n_sim):
        # 小组赛
        group_rankings = simulate_group_stage(rng, groups, team_strengths)
        qualified = []
        for ranking in group_rankings:
            qualified.extend(ranking[:2])

        # 晋级准确率: 固定层级模型按高/中档球队集合计, 随机模型按Top-32计.
        top32_actual = sum(1 for i in range(len(team_strengths))
                           if advancement_mask[i] and i in qualified)
        top32_rates.append(top32_actual / advancement_denominator)

        # 小组赛爆冷: 实力弱的队赢了实力强的队
        for group in [g for g in groups]:
```

```
            indices = [TEAM_INDEX[n] for n in group]
            for a, b in [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]:
                ia, ib = indices[a], indices[b]
                p_i = team_strengths[ia] / (team_strengths[ia] +
                    team_strengths[ib])
                total_group_matches += 1
                # 爆冷: 弱队获胜 (概率 < 0.4)
                if p_i < 0.4:
                    upsets_group += 1 - p_i  # 用期望爆冷度

        # 淘汰赛
        champion, final_ranking = simulate_knockout(rng, qualified,
            team_strengths)
        total_knockout_matches += 31  # 32队淘汰赛 = 31场

        # 冠军实力
        if strength_rank[champion] <= 1:
            champion_top1 += 1
        if strength_rank[champion] <= 4:
            champion_top4 += 1
        if strength_rank[champion] <= 8:
            champion_top8 += 1

        # 完整排名 → Spearman 相关系数
        full_ranking = final_ranking
        group_losers = complete_group_loser_ranking(rng, group_rankings)
        full_ranking = full_ranking + group_losers

        # 计算: 排名位置 vs 实力排名
        pos = list(range(1, 65))
        actual_strength_ranks = [strength_rank[i] for i in full_ranking]
        rho, _ = stats.spearmanr(pos, actual_strength_ranks)
        spearmans.append(rho)

    return {
        "spearman": np.array(spearmans),
        "top32_rate": np.array(top32_rates),
        "champion_top1": champion_top1 / n_sim,
        "champion_top4": champion_top4 / n_sim,
        "champion_top8": champion_top8 / n_sim,
        "upset_rate_group": upsets_group / max(total_group_matches, 1),
        "upset_rate_knockout": upsets_knockout / max(total_knockout_matches
            , 1),
        "advancement_label": advancement_label,
        "advancement_denominator": advancement_denominator,
```

```python
        "advancement_short_label": "高/中档晋级率",
        "champion_mode": "fixed_tier",
    }


# ============================================================
# 3. 替代赛制对比
# ============================================================

def simulate_double_round_robin_group(rng, groups, team_strengths):
    """
    双循环小组赛：每对球队打 2 场（主客场）.
    与单循环对比，增加样本量，减少偶然性.

    [问题] 双循环改为 12 场/组（vs 单循环 6 场/组），更公平但赛程更长.
    """
    group_rankings = []
    for group in groups:
        indices = [TEAM_INDEX[name] for name in group]
        points = defaultdict(int)
        goals_for = defaultdict(int)

        for a, b in [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)]:
            for _ in range(2):  # 主客场各一次
                ia, ib = indices[a], indices[b]
                sa, sb = match_result(rng, team_strengths[ia], team_strengths[ib])
                points[ia] += sa
                points[ib] += sb
                gf_a = generate_goals(rng, team_strengths[ia], team_strengths[ib])
                gf_b = generate_goals(rng, team_strengths[ib], team_strengths[ia])
                goals_for[ia] += gf_a
                goals_for[ib] += gf_b

        ranked = sorted(indices, key=lambda i: (points[i], goals_for[i], rng.random()),
                        reverse=True)
        group_rankings.append(ranked)
    return group_rankings


def compute_double_rr_metrics(rng, groups, team_strengths, n_sim):
    """双循环小组赛 + 单场淘汰"""
```

```
    strength_rank = stats.rankdata([-s for s in team_strengths])
    advancement_mask, advancement_denominator, advancement_label = (
        advancement_reference(strength_rank, team_strengths)
    )
    spearmans = []
    top32_rates = []
    champion_top1 = 0
    champion_top4 = 0
    champion_top8 = 0

    for _ in range(n_sim):
        group_rankings = simulate_double_round_robin_group(rng, groups,
            team_strengths)
        qualified = []
        for ranking in group_rankings:
            qualified.extend(ranking[:2])

        top32_actual = sum(1 for i in range(len(team_strengths))
                           if advancement_mask[i] and i in qualified)
        top32_rates.append(top32_actual / advancement_denominator)

        champion, final_ranking = simulate_knockout(rng, qualified,
            team_strengths)
        if strength_rank[champion] <= 1:
            champion_top1 += 1
        if strength_rank[champion] <= 4:
            champion_top4 += 1
        if strength_rank[champion] <= 8:
            champion_top8 += 1

        group_losers = complete_group_loser_ranking(rng, group_rankings)
        full_ranking = final_ranking + group_losers
        pos = list(range(1, 65))
        actual_strength_ranks = [strength_rank[i] for i in full_ranking]
        rho, _ = stats.spearmanr(pos, actual_strength_ranks)
        spearmans.append(rho)

    return {
        "spearman": np.array(spearmans),
        "top32_rate": np.array(top32_rates),
        "champion_top1": champion_top1 / n_sim,
        "champion_top4": champion_top4 / n_sim,
        "champion_top8": champion_top8 / n_sim,
        "advancement_label": advancement_label,
        "advancement_denominator": advancement_denominator,
```

```
        "advancement_short_label": "高/中档晋级率",
        "champion_mode": "fixed_tier",
    }


def compute_randomized_strength_metrics(
    groups,
    n_sim,
    group_stage_func,
    sigma=LAYER_SIGMA,
    strength_seed=2026,
    match_seed=4200,
):
    """每次赛事先抽取层内正态实力，再模拟小组赛和淘汰赛。"""
    rng_strength = np.random.default_rng(strength_seed)
    rng_match = np.random.default_rng(match_seed)

    spearmans = []
    top32_rates = []
    champion_top1 = 0
    champion_top4 = 0
    champion_top8 = 0

    for _ in range(n_sim):
        team_strengths = draw_layered_strengths(rng_strength, sigma=sigma)
        strength_rank = stats.rankdata([-s for s in team_strengths], method
            ="ordinal")
        advancement_mask, advancement_denominator, advancement_label = (
            advancement_reference(strength_rank, team_strengths)
        )

        group_rankings = group_stage_func(rng_match, groups, team_strengths
            )
        qualified = []
        for ranking in group_rankings:
            qualified.extend(ranking[:2])

        top32_actual = sum(1 for i in range(len(team_strengths))
                           if advancement_mask[i] and i in qualified)
        top32_rates.append(top32_actual / advancement_denominator)

        champion, final_ranking = simulate_knockout(rng_match, qualified,
            team_strengths)
        if strength_rank[champion] <= 1:
            champion_top1 += 1
```

```python
        if strength_rank[champion] <= 4:
            champion_top4 += 1
        if strength_rank[champion] <= 8:
            champion_top8 += 1

        group_losers = complete_group_loser_ranking(rng_match, group_rankings)
        full_ranking = final_ranking + group_losers

        pos = list(range(1, 65))
        actual_strength_ranks = [strength_rank[i] for i in full_ranking]
        rho, _ = stats.spearmanr(pos, actual_strength_ranks)
        spearmans.append(rho)

    return {
        "spearman": np.array(spearmans),
        "top32_rate": np.array(top32_rates),
        "champion_top1": champion_top1 / n_sim,
        "champion_top4": champion_top4 / n_sim,
        "champion_top8": champion_top8 / n_sim,
        "sigma": sigma,
        "advancement_label": "实力Top-32实际晋级比例",
        "advancement_denominator": 32,
        "advancement_short_label": "Top32晋级率",
        "champion_mode": "ranked",
    }


# ============================================================
# 4. 输出
# ============================================================

def print_metrics(m, label, n_sim):
    print(f"\n{'=' * 64}")
    print(f"  {label} ({n_sim} 次模拟)")
    print(f"{'=' * 64}")
    print(f"  Spearman 相关系数 (↑越接近1越公平):")
    print(f"    均值={np.mean(m['spearman']):.4f}  中位={np.median(m['spearman']):.4f}")
    print(f"    5%/95% 分位: {np.percentile(m['spearman'], 5):.4f} / "
          f"{np.percentile(m['spearman'], 95):.4f}")
    print(f"  晋级准确率 ({m.get('advancement_label', '实力Top-32实际晋级比例')}, ↑):")
    print(f"    均值={np.mean(m['top32_rate']):.4f}  "
          f"5%分位={np.percentile(m['top32_rate'], 5):.4f}")
```

```python
    print(f"  冠军实力分布:")
    if m.get("champion_mode") == "fixed_tier":
        print("    固定层级存在并列，仅解释最高实力档夺冠率")
        print(f"    P(最高实力档夺冠)={m['champion_top8']:.4f}")
    else:
        print(f"    P(实力第1名夺冠)={m['champion_top1']:.4f}")
        print(f"    P(实力前4名夺冠)={m['champion_top4']:.4f}")
        print(f"    P(实力前8名夺冠)={m['champion_top8']:.4f}")


def print_comparison_table(title, rows):
    print(f"\n{'=' * 72}")
    print(f"  {title}")
    print(f"{'=' * 72}")
    fixed_tier = all(m.get("champion_mode") == "fixed_tier" for _, m in rows)
    if fixed_tier:
        print(f"  {'赛制':<22} {'Spearman':<10} {'高/中档晋级率':<14} {'最高档夺冠':<10}")
        print(f"  {'-' * 60}")
        for label, m in rows:
            print(f"  {label:<22} {np.mean(m['spearman']):<10.4f} "
                  f"{np.mean(m['top32_rate']):<14.4f} {m['champion_top8']:<10.4f}")
    else:
        print(f"  {'赛制':<22} {'Spearman':<10} {'Top32晋级率':<12} "
              f"{'Top1夺冠':<10} {'Top4夺冠':<10} {'Top8夺冠':<10}")
        print(f"  {'-' * 72}")
        for label, m in rows:
            print(f"  {label:<22} {np.mean(m['spearman']):<10.4f} "
                  f"{np.mean(m['top32_rate']):<12.4f} "
                  f"{m['champion_top1']:<10.4f} {m['champion_top4']:<10.4f} "
                  f"{m['champion_top8']:<10.4f}")


# ============================================================
# MAIN
# ============================================================

def main():
    print("=" * 64)
    print("  浙超赛制评价 — 问题4: 赛制合理化建议")
    print("=" * 64)

```

```python
    # 分组方案: 使用问题1中的ILP均衡方案D
    groups = BALANCED_GROUPS
    c1_ok = len(check_c1(groups)) == 0
    c2_ok = len(check_c2(groups)) == 0
    c3_ok = check_c3(groups)[0] == 0
    print(f"\n  分组方案: Q1-ILP均衡方案D "
          f"C1={'通过' if c1_ok else '失败'} "
          f"C2={'通过' if c2_ok else '失败'} "
          f"C3={'通过' if c3_ok else '失败'}")

    # 实力赋值: 市级=3, 县级市=2, 县=1
    team_strengths = [t.strength for t in TEAMS]
    print(f"\n  实力分布: 市级(3) × {sum(1 for t in TEAMS if t.level=='municipal')} + "
          f"县级市(2) × {sum(1 for t in TEAMS if t.level=='city')} + "
          f"县(1) × {sum(1 for t in TEAMS if t.level=='county')} = {len(TEAMS)} 队")

    n_sim = 5000
    rng = np.random.default_rng(42)

    # --- 当前赛制: 单循环小组赛 + 单场淘汰 ---
    print(f"\n  模拟中... (当前赛制, {n_sim} 次)")
    m_current = compute_metrics(rng, groups, team_strengths, n_sim)
    print_metrics(m_current, "当前赛制: 单循环小组赛 + 单场淘汰", n_sim)

    # --- 替代赛制: 双循环小组赛 + 单场淘汰 ---
    rng2 = np.random.default_rng(42)
    print(f"\n  模拟中... (双循环小组赛, {n_sim} 次)")
    m_double = compute_double_rr_metrics(rng2, groups, team_strengths, n_sim)

    print_metrics(m_double, "双循环小组赛 + 单场淘汰", n_sim)
    print_comparison_table(
        "固定层级实力模型对比 (3/2/1)",
        [("单循环+淘汰", m_current), ("双循环+淘汰", m_double)],
    )

    # --- 层内正态实力扰动: s_i ~ N(level, (1/3)^2) ---
    print(f"\n  模拟中... (层内正态实力扰动 ={LAYER_SIGMA:.3f}, {n_sim} 次)")
    m_rand_current = compute_randomized_strength_metrics(
        groups, n_sim, simulate_group_stage,
        sigma=LAYER_SIGMA, strength_seed=2026, match_seed=4200,
    )
```

```python
    m_rand_double = compute_randomized_strength_metrics(
        groups, n_sim, simulate_double_round_robin_group,
        sigma=LAYER_SIGMA, strength_seed=2026, match_seed=4200,
    )
    print_metrics(m_rand_current, "随机实力: 单循环小组赛 + 单场淘汰", n_sim)
    print_metrics(m_rand_double, "随机实力: 双循环小组赛 + 单场淘汰", n_sim)
    print_comparison_table(
        "层内正态实力模型对比 ( =1/3)",
        [("单循环+淘汰", m_rand_current), ("双循环+淘汰", m_rand_double)],
    )

    # --- 合理化建议 ---
    print(f"\n{'=' * 64}")
    print(f"  赛制合理化建议")
    print(f"{'=' * 64}")
    print()
    print("  1. 当前赛制 (16组×4队单循环 + 32强5轮淘汰):")
    print(f"     - 公平性: Spearman 均值 {np.mean(m_current['spearman']):.3f},")
    print(f"       高/中档球队晋级率 {np.mean(m_current['top32_rate']):.1%}")
    print(f"     - 在随机实力模型下: Top-1/Top-4/Top-8夺冠概率分别为 "
          f"{m_rand_current['champion_top1']:.1%}, "
          f"{m_rand_current['champion_top4']:.1%}, "
          f"{m_rand_current['champion_top8']:.1%}")
    print()
    print("  2. 建议改进:")
    print("     小组赛改为双循环: Spearman 从 "
          f"{np.mean(m_current['spearman']):.3f} 提升至 "
          f"{np.mean(m_double['spearman']):.3f}, "
          f"高/中档晋级率从 {np.mean(m_current['top32_rate']):.1%} 提升至 "
          f"{np.mean(m_double['top32_rate']):.1%}")
    print("       随机实力模型下, Spearman 从 "
          f"{np.mean(m_rand_current['spearman']):.3f} 提升至 "
          f"{np.mean(m_rand_double['spearman']):.3f}, "
          f"晋级准确率从 {np.mean(m_rand_current['top32_rate']):.1%} 提升至 "
          f"{np.mean(m_rand_double['top32_rate']):.1%}")
if __name__ == "__main__":
    main()
```

Listing 6: q4_simulation.py —— Bradley-Terry 胜负模型与层内正态随机实力下的 5 000 次赛事蒙特卡洛仿真

B.5 实验输出文件

正文各表格取数来源如下：

输出文件与对应表格

输出文件	对应表格
`q1_output.txt`	表 3（四种分组方案指标对比）
`strategy_comparison_output.txt`	表 5（朴素贪心与前瞻流 1 000 种子实验）
`generalized_mc_output.txt`	表 6（100 个随机拓扑的广义蒙特卡洛）
`q3_output.txt`	表 7（Haversine 距离选址结果）
`q3_output_v2.txt`	多维交通成本选址实验输出
`q4_output.txt`	表 9（5 000 次赛制仿真结果）